통합반 2회차 — 낭독 대본

[이론 01] 클래스와 인스턴스

[SLIDE 01] 지식 구조의 설계와 그래프 조회

안녕하세요. 이번 회차에서는 설비에 관한 지식을 어떤 구조로 정리하고, 그걸 어떻게 저장해서 찾아 쓰는지 배워 보겠습니다.

쿨러에 이상이 생겼다면 어떤 센서부터 봐야 할까요? 그 센서가 재는 값의 한계는 어디에 적혀 있고, 어떤 원인을 의심해야 할까요? 사람이라면 매뉴얼과 기록을 뒤져서 이 연결을 찾아내겠죠. 그런데 컴퓨터가 같은 길을 따라가려면 얘기가 달라집니다. 설비와 센서, 증상과 원인이 미리 정해진 구조로 연결돼 있어야 해요.

그래서 이번 회차는 이 물음에서 출발합니다. 현장 정보를 클래스와 관계로 나누고, 그 구분을 스키마로 정의한 다음, 그래프 DB에 저장해서 Cypher 쿼리로 직접 찾아보겠습니다.

[SLIDE 02] 학습목표

이번 회차의 학습목표는 화면의 카드 네 장입니다. 첫 카드는 현장의 대상을 클래스와 인스턴스로 나누고, 대상에 적을 값과 다른 대상으로 이어질 관계를 구별하는 거예요. 두 번째 카드는 그 구분을 여러 사람이 함께 쓸 스키마로 적는 일입니다. 식별자와 제약으로 어긋난 데이터를 가려내는 것까지 포함하고요.

세 번째는 정의한 지식을 그래프 DB에 저장하고, Cypher 쿼리로 조회하고 만드는 겁니다. 마지막 카드는 실습이에요. 설비 자료에서 직접 설계한 지식을 그래프에 넣고, 조회 결과가 설계와 맞는지 확인합니다.

용어가 꽤 많아 보이죠. 그런데 전부 하나의 물음으로 이어집니다. 쿨러 사건에 나온 대상을 어떻게 적고, 어떻게 찾느냐는 거예요.

[SLIDE 03] 목차

이번 회차에 갈 길을 목차로 보겠습니다. 위에서부터 여덟 줄이 이론 단원이고, 맨 아래 한 줄이 실습 넷이에요.

처음 세 단원은 대상과 값, 연결을 구별하는 단원입니다. 첫 단원은 유온 센서 하나에서 출발해서 클래스와 인스턴스로 대상을 나누고 관계로 잇습니다. 다음 두 단원에서는 관계 양쪽의 연결 수와 짝에 붙는 값을 본 다음, 클래스를 어떤 기준으로 나눌지 정해요.

다음 세 단원에서는 그 구분을 여러 사람이 함께 쓸 스키마로 적어요. 식별자로 같은 대상을 가리키고, 속성과 연결에 제약을 두고, 설비 자료에서 모델을 직접 설계해 봅니다. 일곱 번째 단원에서는 그래프 DB에 지식이 어떤 모양으로 저장되는지 보고 Cypher 쿼리로 조회하며, 여덟 번째 단원에서는 노드와 관계를 직접 만들어 봅니다.

실습은 이미 있는 그래프를 조회하고 지식 지도를 탐색하는 데서 시작합니다. 그다음에 설비 자료로 직접 설계하고, 그걸 그래프에 구현해서 확인하겠습니다.

먼저 대상과 값, 연결을 구별하는 세 단원부터 살펴보겠습니다.

[SLIDE 04] 클래스 · 인스턴스와 관계의 기초

첫 단원의 출발점은 유온 센서 하나에 관한 지식을 어떻게 적느냐입니다. 쿨러에 이상이 생겨서 유온 센서부터 찾아본다고 해 볼게요. 그런데 거기에 센서 이름만 달랑 적혀 있다면 어떨까요? 이 센서가 무슨 값을 내놓는지, 어느 장치의 상태를 보는지는 알 수가 없겠죠.

그래서 이 단원에서는 센서에 관한 지식을 하나씩 채워 보겠습니다. 먼저 이게 어떤 종류의 대상인지 정하고, 이름과 단위 같은 값을 채웁니다. 그다음에 이 센서가 재는 물리량, 그리고 주변 장치와 연결해요. 다 적고 나면 센서 하나에서 출발해서 쿨러의 상태와 정비에 필요한 정보까지 찾아갈 수 있게 됩니다.

이 단원에서 세우는 말, 클래스와 인스턴스, 속성과 관계는 이번 회차 끝까지 계속 씁니다. 뒤에서 스키마를 적을 때도, 그래프에서 쿼리로 찾을 때도 결국 이 말들로 적거든요. 그래서 처음에 센서 하나로 차근차근 구별해 두려는 겁니다.

[SLIDE 05] 클래스와 인스턴스

종류를 나누는 얘기는 설비 목록에서 시작해 볼게요. 화면 오른쪽 칸을 보면 이름이 다른 유압 설비 세 대가 있습니다. 경보가 났을 때 어느 설비에서 난 건지 알려면, 이 세 대는 서로 구별돼야겠죠. 그런데 세 대 모두 설비라는 공통점도 있어요. 이 공통점과 하나하나의 대상을 나눠 부르는 말이 클래스와 인스턴스입니다.

왼쪽 칸이 클래스예요. 클래스는 공통 특성을 가진 대상의 유형입니다. 여기서는 설비가 클래스고요. 설비라면 어떤 정보를 적고 어떤 대상과 연결할지를, 클래스가 공통으로 정해 둡니다.

오른쪽 칸이 인스턴스입니다. 그 유형에 속하는 개별 대상 하나하나를 말해요. 첫 번째 설비도 인스턴스고, 두 번째 설비도 또 다른 인스턴스죠. 같은 정의를 쓰면서, 이름과 내용은 각자 따로 갖는 겁니다.

이 구분을 센서에도 대 볼게요. 오른쪽 칸 아래쪽에 유온 센서와 냉각 효율 채널이 있죠. 둘 다 값을 내놓는 계측 채널이라서 센서라는 클래스에 속합니다. 그렇다고 둘이 같은 대상은 아니잖아요. 그래서 유온 센서도 인스턴스 하나, 냉각 효율 채널도 인스턴스 하나로 따로 둡니다. 오른쪽 칸 맨 아래 문장도 읽어 보세요. 「유온 센서는 센서의 하나이다」. 인스턴스와 클래스의 사이는 이렇게 「무엇의 하나」라고 읽으면 됩니다.

이 구분은 센서를 더 등록할 때 바로 쓰입니다. 이름이 다르다고 센서라는 유형을 매번 새로 만들 필요는 없어요. 공통으로 적을 항목은 같은 정의를 쓰고, 새 센서의 내용만 채우면 됩니다.

찾을 때도 이 구분이 그대로 쓰여요. 센서를 전부 보여 달라는 건 클래스로 묻는 거고, 유온 센서의 단위가 뭐냐는 건 인스턴스 하나를 묻는 거죠. 어느 쪽을 묻느냐에 따라 찾아야 할 범위가 달라지는 거예요.

[SLIDE 06] 센서 클래스의 속성과 속성값

그럼 유온 센서에 정보를 채워 보죠. 이름을 적고, 계측 데이터를 찾을 때 쓰는 태그를 적고, 단위도 적습니다. 이렇게 값을 담는 항목 하나하나를 속성이라고 해요. 그리고 그 항목에 실제로 채워 넣은 내용이 속성값입니다.

표를 보면 맨 왼쪽 열이 센서 클래스가 정한 공통 항목이에요. 가운데 열은 유온 센서, 오른쪽 열은 냉각 효율 채널의 값이고요. 같은 가로줄끼리 비교해 보면 됩니다.

셋째 줄, 태그 줄을 볼까요. 가운데 적힌 TS1은 유온 센서의 태그 칸에 채운 값입니다. 바로 옆 냉각 효율 채널에도 태그 칸이 똑같이 있고, 거기에 자기 값을 갖고 있죠. 그러니까 프로그램은 어느 센서를 고르든 같은 태그 항목을 읽어서 계측 데이터를 찾을 수 있습니다.

그 아래 단위 줄도 마찬가지예요. 유온은 온도라서 섭씨, 냉각 효율은 비율이라서 퍼센트죠. 단위는 다르지만, 그렇다고 표의 구조가 달라지지는 않습니다. 같은 항목을 서로 다른 값으로 채웠을 뿐이에요. 맨 위 식별자와 이름도 각 채널을 구별하는 정보고요.

마지막 줄은 값을 어떻게 얻느냐입니다. 유온 센서는 온도를 직접 재요. 냉각 효율 채널은 다른 계측값으로 계산해서 값을 내놓고요. 그래서 계산으로 값을 얻는 채널인지를 참과 거짓으로 적어 둔 겁니다. 직접 재든 계산하든, 두 채널 모두 태그와 단위를 갖고 값을 내놓으니까 센서 클래스로 함께 다룹니다.

[SLIDE 07] 센서와 상태 변수

센서 칸을 다 채웠는데도 아직 답을 못 하는 물음이 있습니다. 유온이 너무 높아졌는지 판단하려면 뭘 찾아야 할까요? 센서 이름이 필요한 게 아니죠. 기름 온도를 어디까지 허용하는지가 필요합니다. 화면 오른쪽에 실습 설비에 넣어 둔 두 기준이 보이죠. 정상값은 50도 조금 아래, 한계는 65도예요. 그런데 이 기준이 센서가 아니라 유온 쪽에 붙어 있어요. 왜 그렇게 했을까요?

왼쪽 유온 센서는 재는 쪽입니다. 어느 계측 채널인지, 계측 데이터를 어떤 태그로 찾는지가 여기 담겨 있어요.

오른쪽 유온은 그 센서가 재는 물리량이에요. 이런 물리량을 상태 변수라고 부릅니다. 기름 온도나 팬 속도 같은 게 상태 변수죠. 그리고 유온에는 평소에 어느 정도인지를 나타내는 정상값과 한계가 붙습니다. 이 한계는 앞 회차에서 본 인터록, 곧 PLC가 설비를 보호하려고 정해 둔 경계예요.

정비하다가 온도를 재는 센서를 바꾼다고 생각해 보세요. 재는 장치는 달라졌지만, 우리가 관리하려는 건 여전히 같은 기름 온도입니다. 센서가 바뀌었다고 허용할 온도까지 바뀌지는 않겠죠.

같은 기름 온도를 재는 센서가 둘이라고 해도 마찬가지예요. 기준을 센서마다 적어 두면 한계도 두 군데 적히겠죠. 그러다 한쪽만 고치면, 같은 유온에 한계가 둘이 돼 버립니다. 기준을 유온 한 곳에 두면, 센서 몇 개가 읽든 비교할 기준은 하나로 남아요.

화면의 두 대상은 이름은 비슷해도 하는 일이 이렇게 다릅니다. 그래서 무엇으로 재는지와 어떤 물리량을 관리하는지를 따로 적어요. 유온 센서를 찾으면 계측 채널의 정보를 얻고, 유온을 찾으면 그 물리량의 기준을 얻는 겁니다.

[SLIDE 08] 데이터 속성과 객체 간 관계

유온 센서의 태그를 알고 싶다면 센서의 칸 하나만 읽으면 됩니다. 그런데 유온의 한계를 알고 싶다면 어떨까요? 센서에서 유온이라는 대상으로 한 번 건너간 다음, 거기서 한계 칸을 읽어야 하죠. 값을 읽는 일과 다른 대상을 찾아가는 일이 여기서 갈립니다.

화면 왼쪽이 데이터 속성입니다. 앞에서 채운 속성이 바로 이거예요. 인스턴스에 값 하나를 붙이는 쪽이죠. 센서의 태그와 단위가 여기에 해당하고, 유온에 붙은 한계도 데이터 속성입니다. 어느 대상의 어느 항목인지만 정하면 값이 바로 나오죠.

오른쪽은 객체 간 관계입니다. 한 인스턴스를 다른 인스턴스에 잇는 방법이에요. 유온 센서와 유온을 연결하고, 그 연결을 「읽는다」로 정했습니다. 그러면 이 센서가 읽는 대상이 유온이라는 걸 찾을 수 있고요. 오른쪽 칸 마지막 줄처럼, 건너간 유온에서 한계를 읽으면 됩니다.

여기서 헷갈리기 쉬운 게 있어요. 둘을 가르는 기준은 글자냐 숫자냐가 아닙니다. 태그는 글자로 적은 속성값이죠. 유온도 이름은 글자로 적지만, 자기 정보를 가진 하나의 대상입니다. 필요한 정보가 센서의 칸 하나에 들어 있는지, 연결된 다른 대상에 있는지를 보면 구분이 돼요.

이제 유온을 왜 따로 대상으로 뒀는지 보입니다. 센서 옆에 「유온」이라고 글자만 적어 둬도, 사람은 뭘 재는지 짐작할 수 있어요. 하지만 지금 필요한 건 이름 확인에서 끝나지 않죠. 그 물리량의 기준까지 찾아야 합니다. 그래서 유온이라는 인스턴스에 기준을 모으고, 센서를 거기에 연결한 거예요.

[SLIDE 09] 설비 · 구성 요소 · 부품

유온 센서가 무엇을 재는지는 정리가 됐습니다. 다음 물음은 이 센서가 어느 장치의 상태를 보느냐예요. 그 장치가 오일 쿨러입니다. 화면을 보면 오일 쿨러와 쿨러 코어를 서로 다른 종류로 나눠 놓았죠.

맨 위 설비부터 볼게요. 설비는 유압 파워팩 한 대를 가리킵니다. 가운데 구성 요소는 그 설비를 이루는 장치예요. 오일 쿨러처럼 설비 안에서 하나의 역할을 맡는 장치죠.

맨 아래 부품은 그 장치 안에서 갈아 끼우는 교체품입니다. 사실 일상에서는 장치도 부품이라고 부르곤 하죠. 그런데 이 지식 모델에서는 역할로 구별해서 적습니다. 상태를 살피는 장치는 구성 요소, 그 안에서 교체하는 건 부품이에요.

이렇게 나누면 뭐가 좋을까요? 오일 쿨러를 정비하면서 쿨러 코어를 주문한다고 해 보죠. 그때 찾아야 하는 건 장치 전체의 이름이 아니라 교체할 부품과 그 부품 번호입니다. 그래서 쿨러 코어를 부품으로 따로 둬요. 반대로 쿨러가 제대로 식히고 있는지 볼 때는, 구성 요소인 오일 쿨러에서 계측 채널을 찾습니다. 유온 센서와 냉각 효율 채널이 이 장치의 상태를 보여 주죠.

그러니까 「쿨러」라는 말만 보고 장치 전체인지 교체품인지 다시 짐작할 일이 줄어듭니다. 교체품을 고르는 일과 상태를 살피는 일이 서로 다른 대상으로 이어지는 거예요.

[SLIDE 10] 증상 · 고장 유형 · 원인

오일 쿨러 상태를 봤더니 유온은 오르고 냉각 효율은 떨어지고 있습니다. 이걸 기록할 때는 두 가지를 나눠야 해요. 무엇이 보였는가, 그리고 왜 그렇게 됐는가.

맨 위가 증상입니다. 관측한 이상 양상이에요. 유온 상승, 냉각 효율 저하가 여기 들어가죠.

가운데는 고장 유형입니다. 이 증상을 보고 살펴볼 기능상의 문제예요. 여기서는 쿨러의 냉각 성능 상실이고요. 어느 구성 요소에서 어떤 고장이 생기는지를 나타냅니다.

맨 아래가 원인입니다. 그 고장이 왜 생겼느냐죠. 쿨러 핀 오염이 원인 후보고, 팬 구동 불량이나 주변 온도 상승도 후보가 됩니다. 여기서 하나 기억해 둘 게 있어요. 유온이 올랐다는 걸 확인했다고 해서, 이 후보 가운데 하나로 원인이 정해진 건 아닙니다. 원인을 가리려면 증거가 더 필요하죠.

쿨러 사건에서 에이전트가 걸은 길도 바로 이 순서였어요. 경보에서 출발해 증상과 고장 유형에 닿고, 거기서 원인 후보 셋을 찾은 다음, 후보마다 증거가 되는 값을 확인해서 가렸죠. 세 칸이 나뉘어 있었기 때문에, 이미 관측한 사실과 아직 확인해야 할 후보가 섞이지 않았던 겁니다.

이름을 적을 때도 같은 구분을 씁니다. 유온 상승이라는 인스턴스는 증상 클래스에, 쿨러 냉각 성능 상실은 고장 유형 클래스에, 핀 오염은 원인 클래스에 둬요. 손에 잡히는 물건만 인스턴스가 되는 건 아닙니다. 어떤 증상이 어떤 고장을 가리키는지, 무엇이 그 고장을 일으키는지 연결하려면, 이런 대상도 하나씩 구별해 둬야 하거든요.

[SLIDE 11] 관계 유형의 이름과 방향

지금까지 나눈 대상들을 이번엔 화살표로 이어 보겠습니다. 왼쪽 위 설비에서 출발해서 따라가 볼게요. 설비는 구성 요소를 갖습니다. 구성 요소는 센서로 계측되고요. 센서는 상태 변수를 읽습니다.

동그라미 안에는 클래스 이름이 들어 있어요. 실제 대상으로 바꿔 읽으면 유압 설비, 오일 쿨러, 유온 센서, 유온, 이렇게 이어집니다. 선 옆의 작은 이름은 그 연결이 무슨 뜻인지를 나타내고요. 이 이름들은 영어 대문자로 적혀 있죠. 그래프에 실제로 저장된 관계의 이름이 바로 이거예요. 뒤에서 쿼리를 적을 때도 이 이름을 한 글자도 바꾸지 않고 그대로 씁니다.

선 하나를 이을 때는 이름만 정하는 게 아닙니다. 어느 쪽에서 시작해서 어느 쪽으로 향하는지도 정하고, 양 끝에 어떤 클래스가 올 수 있는지도 정해요. 이렇게 이름과 방향, 시작과 끝의 클래스를 한꺼번에 정한 것을 관계 유형이라고 합니다.

가운데, 오일 쿨러와 유온 센서를 잇는 선을 보죠. 「쿨러는 센서로 계측된다」는 뜻이라서 구성 요소에서 센서 쪽으로 향합니다. 그런데 이 선 끝에 교체 부품을 갖다 놓으면 어떻게 될까요? 이 관계가 정한 뜻과 맞지 않게 되죠. 교체품을 이으려면 거기에 맞는 다른 관계가 필요합니다.

관계 유형을 이렇게 공통으로 정해 두면, 다른 장치에서도 같은 뜻으로 읽을 수 있어요. 어떤 구성 요소든 같은 계측 관계를 따라가면 센서가 나오는 거죠. 다만 실제로 이을 대상은 하나하나 채워야 합니다. 유온 센서는 유온을 읽고, 냉각 효율 채널은 냉각 효율을 읽도록 연결하는 식이에요. 공통으로 정한 연결 방식과 실제로 이은 대상, 이 둘이 함께 있어야 지식을 따라갈 수 있습니다.

[SLIDE 12] 조작 관계와 물리적 영향의 구분

관계로 나타낼 수 있는 지식이 하나 더 있습니다. 쿨러 사건에서 팬을 빠르게 돌려서 유온을 낮췄던 거 기억하시죠? 이번엔 팬을 움직이는 일과, 그 결과 기름이 식는 일을 나눠 보겠습니다.

왼쪽이 조작 관계입니다. 팬 인버터는 구동기예요. 구동기는 PLC가 값을 써서 움직이는 장치를 말합니다. 이 장치가 직접 바꾸는 건 팬 속도죠. 그래서 팬 인버터에서 팬 속도로 「바꾼다」는 관계를 둡니다.

오른쪽은 팬 속도를 바꾼 다음에 생기는 물리적 영향이에요. 팬 속도가 올라가면 냉각이 늘고, 유온은 내려가는 쪽으로 영향을 받습니다. 그래서 팬 속도에서 유온으로 「영향을 준다」는 관계를 둬요.

왜 굳이 둘로 나눌까요? 이 예에서 명령으로 직접 값을 써 넣는 쪽은 구동기가 바꾸는 팬 속도예요. 유온은 명령으로 바로 정할 수 없고, 영향을 받아 따라오는 값이죠. 그래서 무엇을 움직일 수 있는지는 조작 관계에서 찾고, 그걸 움직이면 무엇이 어느 쪽으로 변하는지는 영향 관계에서 찾습니다. 조치를 고를 때 이 두 물음은 따로 답해야 하거든요.

오른쪽 칸 아래, 이 영향 관계에 붙은 마이너스 1을 보세요. 앞의 값이 오르면 뒤의 값은 내려간다는 방향을 뜻합니다. 이렇게 연결 그 자체에 붙이는 값을 관계의 속성이라고 해요.

맨 아래 견줄 것도 보세요. 펌프 부하에서 유온으로 가는 영향 관계에는 플러스 1이 붙어 있습니다. 부하가 오르면 발열이 늘어서 유온도 오르는 쪽이죠. 같은 유온으로 들어오는 영향이라도, 유온을 어느 쪽으로 움직이는지는 이 부호로 가립니다.

여기서 두 가지를 조심해야 합니다. 먼저 화살표와 부호는 서로 다른 걸 말해요. 화살표는 영향을 주는 쪽에서 받는 쪽으로 갑니다. 부호가 마이너스라고 해서 화살표를 유온에서 팬 속도로 뒤집지는 않아요.

그리고 마이너스 1은 온도를 1도 낮추라는 명령이 아닙니다. 유온이 얼마나 내려가는지 계산한 값도 아니고요. 이 관계에 적은 지식은 두 물리량이 변하는 방향, 그것뿐입니다.

[SLIDE 13] 한 대상의 들어오는 관계와 나가는 관계

정비 담당자가 되어 오일 쿨러를 살펴본다고 생각해 보죠. 알고 싶은 건「이 장치에 어떤 고장이 생길 수 있나」예요. 화면 가운데 구성 요소를 오일 쿨러라고 놓고, 어느 선을 보면 될지 찾아보세요.

답은 왼쪽 아래, 고장 유형에서 쿨러로 들어오는 선입니다.「고장 유형이 이 구성 요소에서 발생한다」는 뜻이에요. 쿨러에서 출발해서 이 선을 거슬러 가면 쿨러 냉각 성능 상실이 나옵니다. 화살표가 쿨러에서 나가지 않아도 찾을 수 있는 거죠.

왼쪽 위의 선은 설비가 구성 요소를 갖는다는 관계입니다. 이것도 쿨러로 들어오는 선이에요. 거슬러 가면 쿨러가 어느 설비에 연결돼 있는지 알 수 있습니다.

오른쪽은 쿨러에서 나가는 관계들이에요. 위쪽 계측 관계를 따라가면 상태를 볼 센서가 나오고요. 아래쪽 부품 관계를 따라가면 교체품인 쿨러 코어가 나옵니다. 같은 쿨러에서 출발해도, 무엇을 알고 싶으냐에 따라 따라갈 선이 달라지는 거죠.

하나 짚고 갈게요. 들어오는 선을 거슬러 본다고 해서 반대 뜻의 관계를 새로 만드는 건 아닙니다. 이미 저장된 관계를 반대편에서 찾아보는 것뿐이에요.

쿨러 성능 저하 경보에서 출발한 길도 이 그림에서 이어집니다. 앞에서 본 것처럼 경보에서 증상을 거쳐 고장 유형에 닿았다면, 그 고장이 발생하는 구성 요소로 건너가 쿨러에 닿습니다. 거기서 나가는 부품 관계를 따라가면 교체품인 쿨러 코어까지 가고요.

이제 단원 처음의 물음에 답할 수 있습니다. 센서 이름만으로는 뭘 재는지, 어느 장치의 상태를 보는지 알기 어려웠죠. 지금은 다릅니다. 센서의 속성에 태그와 단위가 있고, 관계를 따라가면 재는 유온과 그 한계, 상태를 보는 쿨러까지 닿아요. 쿨러에서 나가는 관계와 들어오는 관계를 함께 보면 고장 유형과 교체품도 찾을 수 있고요. 대상을 나누고, 값을 채우고, 관계를 잇는 일이 이렇게 질문 하나에 답하는 길을 만들어 줍니다.

[SLIDE 14] 카디널리티와 관계의 속성

첫 단원에서 대상을 나누고 관계로 이어 봤죠. 두 번째 단원에서는 그 관계를 한 걸음 더 정밀하게 정해 봅니다. 먼저 이런 물음이 있어요. 관계 한쪽에 대상이 몇 개까지 이어질 수 있을까요? 쿨러 하나에 센서를 하나만 둘 수 있는지, 여러 개 둘 수 있는지도 정의에 들어가야겠죠. 또 하나는, 부품과 공급사처럼 짝마다 값이 달라질 때 그 값을 어디에 두느냐입니다. 연결의 수를 정하는 카디널리티를 먼저 보고, 연결 하나에 붙는 값은 그다음에 보겠습니다.

[SLIDE 15] 카디널리티의 네 가지 형태

팬 인버터와 팬 속도가 연결돼 있습니다. 여기에 다른 상태 변수를 하나 더 연결해도 될까요? 선은 얼마든지 그릴 수 있죠. 하지만 선을 그릴 수 있다는 것과 그 연결이 정의에 맞는다는 건 다른 얘기입니다. 관계 양쪽에 대상이 몇 개씩 이어질 수 있는지를 정한 것, 그게 카디널리티예요.

화면에서 하나는 숫자 1로, 여럿은 N이나 M으로 표시했습니다. 칸마다 왼쪽 흰 원이 관계의 시작 쪽, 오른쪽 파란 원이 끝 쪽이에요. 읽을 때는 양쪽을 다 봐야 합니다. 왼쪽 대상 하나에서 오른쪽을 보고, 다시 오른쪽 대상 하나에서 왼쪽을 보는 거죠.

첫 칸은 일대일입니다. 이 정의에서는 구동기 하나가 상태 변수 하나를 바꾸고, 반대로 상태 변수 쪽에서도 구동기 하나만 대응해요. 팬 인버터와 팬 속도가 그 예입니다. 이렇게 정해 두면 팬 속도를 바꾸고 싶을 때 값을 써 넣을 장치가 하나로 정해지죠.

두 번째 칸은 일대다예요. 원인 하나에 증거가 여럿 이어질 수 있습니다. 쿨러 핀 오염에는 증거가 둘 붙어 있죠. 냉각 효율 평균이 낮다는 것과 팬 진동이 정상이라는 것입니다. 앞 회차의 쿨러 사건에서 핀 오염을 가릴 때 확인한 값이 바로 이 둘이었어요. 반대로 증거 쪽에서 보면 딸린 원인은 하나입니다. 그리고 둘은 지금 들어 있는 증거 수일 뿐이에요. 일대다는 증거를 여럿 둘 수 있다는 정의지, 다른 원인에도 꼭 둘을 두라는 뜻이 아닙니다.

세 번째 칸은 다대일입니다. 센서가 상태 변수를 읽는 관계죠. 센서 쪽에서 보면 읽는 상태 변수는 하나고, 상태 변수 쪽에서 보면 센서가 여럿일 수 있습니다. 지금 데이터에서는 유온을 읽는 센서가 유온 센서 하나뿐이에요. 그래도 같은 기름 온도를 재는 센서를 하나 더 달면 이 정의에 그대로 맞습니다. 앞에서 센서가 둘이어도 기준은 유온 한 곳에 둔다고 했던 게 바로 이 모양이에요.

마지막 칸은 다대다예요. 부품 하나를 여러 공급사가 공급할 수 있고, 공급사 하나도 여러 부품을 공급할 수 있습니다. 어느 쪽에서 봐도 상대가 여럿일 수 있으니까 다대다입니다. 앞에서 본 원인과 고장 유형 사이도 다대다예요. 쿨러 냉각 성능 상실 하나에 원인 후보가 셋 붙어 있었죠. 반대로 원인 하나가 다른 고장을 일으킬 수도 있고요.

새 공급사를 등록하는 장면으로 생각해 볼까요. 이미 공급사가 있는 부품에 다른 회사를 하나 더 연결하려고 합니다. 다대다로 정해 뒀다면 기존 회사를 지우지 않고 새 회사를 함께 연결하면 돼요. 그런데 일대일로 정해 뒀다면 같은 입력이 정의에 맞지 않게 되겠죠. 그러니까 카디널리티는 그림 속 선의 개수를 설명하는 데서 끝나지 않습니다. 앞으로 어떤 연결을 허용할지 정하는 기준이 되는 거예요.

[SLIDE 16] 카디널리티 선언과 실제 데이터

카디널리티를 읽을 줄 알게 됐으니, 실제 연결과도 맞춰 보겠습니다. 화면 왼쪽은 잘못 연결한 상황을 가정한 거고요, 오른쪽은 지금 지식 그래프에 실제로 들어 있는 구조입니다.

왼쪽부터 보죠. 조작 관계는 일대일로 선언돼 있습니다. 그런데 팬 인버터에서 팬 속도와 펌프 부하로 선이 두 개 나갔어요. 시작 대상 하나에 끝 대상이 둘 붙었으니까 선언과 맞지 않습니다. 이럴 땐 어느 연결을 잘못 넣었는지 확인해야겠죠.

오른쪽은 설비가 구성 요소를 갖는 관계예요. 선언은 일대다입니다. 그런데 실제로는 같은 기종의 세 설비가 오일 쿨러 노드 하나를 함께 가리키고 있어요. 쿨러 쪽에서 거꾸로 보면 설비가 여럿이죠. 그러니까 이 실제 연결은 일대다라는 선언과 다릅니다.

그렇다고 쿨러 한 대를 세 설비가 같이 쓴다는 뜻은 아니에요. 이 그래프는 같은 기종에 공통인 구성 지식을 노드 하나로 적은 겁니다. 세 설비가 같은 센서와 교체 부품 정보를 찾아가도록 한 거죠.

그럼 오른쪽 같은 경우는 어떻게 할까요? 선 하나를 바로 지우기보다, 이 모델이 무엇을 나타내려는 건지부터 확인해야 해요. 이 모델은 같은 기종에 공통인 구성 지식을 노드 하나로 두는 쪽을 골랐습니다. 다만 이 의도가 있다고 해서 일대다 선언과 실제 연결이 맞아지는 건 아닙니다. 공통 구성 지식을 함께 가리키려는 설계가 맞다면 그에 맞게 선언을 정리해야 하고, 설비별 대상을 나타내려는 거라면 데이터를 나눠야겠죠. 지금 화면은 이 불일치가 남아 있는 사례로 보시면 됩니다. 만약 설비마다 쿨러의 교체 이력을 따로 관리해야 한다면, 그때는 대상의 범위부터 다시 정해서 설비마다 쿨러를 구별할 수 있게 해야 합니다.

그리고 기억해 둘 게 있습니다. 카디널리티 선언을 적었다고 해서 데이터베이스가 이런 연결을 알아서 막아 주지는 않아요. 실제로 저장된 연결을 세어 보고, 그 노드가 무엇을 대표하는지까지 같이 대조해야 합니다. 선언은 검사할 기준이고, 실제 데이터가 그 기준을 따르는지는 따로 확인할 일이에요.

[SLIDE 17] 두 대상의 짝에 붙는 속성

같은 씰 키트를 공급사 여러 곳이 공급하고 있습니다. 부품은 같은데 단가도 납기도 다 다르네요. 그럼 이 값을 부품에 적어야 할까요, 공급사에 적어야 할까요? 화면 아래 세 줄을 보면, 부품이나 공급사 한쪽만 골라서는 공급 조건을 하나로 정할 수가 없습니다.

부품에 단가를 하나 적는다고 해 보죠. 세 회사 중 누구 가격을 적어야 할지 정할 수가 없어요. 반대로 공급사에 단가를 하나 적으면, 그 회사가 파는 다른 부품 가격과 섞여 버립니다. 부품과 공급사를 함께 골랐을 때 비로소 공급 조건 하나가 정해지는 거죠.

이렇게 두 대상의 짝에 딸린 값은 관계의 속성으로 둡니다. 앞에서 본 팬 속도와 유온 사이의 부호처럼, 연결 자체에 붙이는 값이에요. 화면 위쪽이 공급 관계의 정의고, 아래쪽이 씰 키트와 세 공급사 사이에 실제로 적힌 값입니다. 정의를 보면 단가와 불량률은 필수, 납기는 선택이에요. 단가는 만원 단위, 납기는 일 단위, 불량률은 비율로 적었습니다.

아래 세 줄에서 같은 항목끼리 위아래로 견주어 보세요. 공급사가 바뀌면 값도 바뀌죠. 단가가 가장 싼 곳은 납기도 가장 빠르지만, 불량률은 가장 높습니다. 가장 비싼 곳은 불량률이 가장 낮은 대신 납기가 가장 길고요. 어느 공급사가 나은지는 값 하나만 봐서는 정할 수 없고, 세 값을 함께 놓고 따져야 하는 거예요. 그런 비교가 되려면 값이 짝마다 따로 적혀 있어야 합니다. 가격이 달라지는 자리도, 납기가 달라지는 자리도 모두 부품과 공급사의 연결이니까요.

이 구조는 공급 조건이 바뀌어도 그대로 쓸 수 있어요. 한 회사가 씰 키트 가격을 바꿨다면 그 짝의 단가만 고치면 됩니다. 다른 회사의 씰 키트 가격이나 같은 회사의 쿨러 코어 가격은 건드릴 필요가 없죠. 불량률도 같은 식으로 읽어야 합니다. 이 값은 그 회사가 이 부품을 공급할 때의 불량률이지, 회사 전체의 불량률이 아니에요.

참고로 화면의 첫 두 공급사는 씰 키트 말고도 팬 베어링과 쿨러 코어를 공급하고, 마지막 공급사는 이 가운데 씰 키트만 공급합니다. 그래서 부품 셋과 공급사 셋 사이에 공급 관계가 여럿 생깁니다. 다대다는 이런 연결들을 허용하는 모양이고, 관계의 속성은 그중 연결 하나를 골랐을 때 읽는 조건입니다.

[SLIDE 18] 클래스 계층과 모델링 선택

연결의 수와 짝에 붙는 값까지 정했으니, 세 번째 단원에서는 모델을 무엇으로 나눌지 정합니다. 지금까지 센서와 구동기는 클래스를 나눴고, 유온 센서와 냉각 효율 채널은 같은 센서 클래스에 넣었죠. 그럼 이런 선택을 직접 해야 할 때는 무엇을 기준으로 삼을까요? 공통 정의를 물려받는 상속을 먼저 보고, 새 클래스를 만들 때와 기존 정의에 값을 채울 때를 구별해 보겠습니다.

[SLIDE 19] 클래스 계층과 상속

업무 흐름을 떠올려 보면, 시작이나 끝을 알리는 지점이 있고, 수행할 작업이 있고, 길이 갈라지는 지점도 있습니다. 하는 일은 서로 다르지만 모두 업무 흐름의 한 지점이라는 공통점이 있죠. 그림 맨 위의 FlowNode가 바로 이 공통 유형입니다.

이렇게 공통 내용을 정한 위쪽을 상위 클래스, 그 정의를 물려받는 아래쪽을 하위 클래스라고 합니다. 여기서는 이벤트, 작업, 게이트웨이가 하위 클래스예요. 상위 클래스가 정한 식별자와 이름 속성을 셋 모두 물려받습니다. 이걸 상속이라고 해요.

가운데 작업 클래스를 보세요. 물려받은 공통 속성에 작업 종류를 하나 더합니다. 그러니까 규정 검토라는 작업 하나를 등록하면, 식별자와 이름도 갖고 어떤 작업인지 나타내는 값도 갖게 되는 거죠. 공통 정의를 세 군데 다시 적을 필요가 없습니다.

그리고 선 끝에 빈 세모가 보이죠. 이 세모가 상위 클래스를 가리킵니다. 센서가 물리량을 읽는다는 일반 관계와는 달라요. 작업이 흐름 노드의 한 종류라는 뜻을 나타내는 표기입니다.

실제 규정 검토 노드에는 Task와 FlowNode, 이름표 두 개가 함께 붙습니다. 작업 하나를 둘로 복사한 게 아니에요. 그 대상이 작업이면서 동시에 흐름 노드라는 걸 나타낸 겁니다. 그래서 작업만 찾을 때도, 업무 흐름의 모든 지점을 찾을 때도 같은 대상이 각각의 범위에 들어가죠. 반대로 흐름 노드라는 이름표가 빠져 있다면, 업무 흐름의 지점을 모두 찾을 때 그 작업만 빠지게 됩니다.

여기서 헷갈리면 안 되는 게 있어요. 오일 쿨러는 구성 요소 클래스의 하위 클래스가 아니라 인스턴스입니다. 종류를 새로 정의한 게 아니라, 그 종류에 속하는 대상 하나를 적은 거니까요. 또 구성 요소가 설비 안에 있다고 해서 설비의 하위 클래스가 되는 것도 아닙니다. 장치가 설비의 한 종류는 아니잖아요. 포함되는 사이는 관계로 잇고, 종류끼리 공통 정의를 물려받는 사이는 상속으로 나타냅니다. 그래서 이 모델에서 상속이 쓰인 곳은 지금 본 업무 흐름 쪽 한 곳뿐이에요. 설비 쪽 클래스들은 어느 하나가 다른 것의 한 종류가 아니라, 갖는다, 계측한다, 부품을 쓴다 같은 관계로 이어지기 때문입니다.

[SLIDE 20] 새 클래스를 정의하는 기준

설비에 장치를 하나 더 등록한다고 해 보죠. 기존 센서 정의에 넣으면 될까요, 아니면 새 클래스를 만들어야 할까요? 기준은 장치 이름이 아닙니다. 어떤 정보를 갖고, 무엇과 어떤 관계를 맺느냐예요.

왼쪽 칸에 센서와 구동기가 같이 있습니다. 센서는 값을 읽어요. 태그로 계측 데이터를 찾고, 상태 변수와는 읽는 관계로 연결돼 있죠. 구동기는 앞에서 본 것처럼 값을 써서 움직이는 쪽이라, 바꿀 설정값과 허용 범위를 갖고 상태 변수를 바꾸는 관계로 연결돼요.

구동기 쪽에 적힌 최소와 최대를 보세요. 숫자라서 유온의 정상값이나 한계와 비슷해 보이지만 뜻이 다릅니다. 구동기의 범위는 우리가 써 넣을 수 있는 설정값의 범위고, 유온의 기준은 재어지는 물리량을 비교할 값이에요. 숫자라고 해서 같은 칸으로 섞지 않는 이유입니다.

쿨러 사건에 대 보면 차이가 분명해집니다. 기름 온도를 확인할 때는 유온 센서의 태그가 필요하죠. 팬 속도를 바꿀 때는 팬 인버터가 받는 설정값과 허용 범위가 필요하고요. 읽는 장치의 정보만으로는 조작 명령을 만들 수 없습니다. 속성도 관계도 다르니까 클래스를 나누는 거예요.

오른쪽 칸을 보세요. 유온 센서, 토출 압력 센서, 팬 진동 센서는 재는 물리량도 단위도 다릅니다. 그런데 태그와 단위를 갖고 상태 변수를 읽는 구조는 똑같죠. 그러니까 기존 센서 클래스에 인스턴스를 더하면 됩니다.

오른쪽 칸 마지막 줄에 붙은「이 모델의 질문 범위에서는」이라는 단서도 보세요. 센서마다 클래스를 두면, 센서가 늘 때마다 클래스도 늘고 센서 전체를 한 번에 찾기도 어려워집니다. 그렇다고 그게 언제나 틀렸다는 뜻은 아니에요. 지금 이 모델이 답하려는 질문에서는 나눌 까닭이 없다는 겁니다.

판단이 애매하면, 그 차이를 속성값으로 표현할 수 있는지부터 보세요. 온도와 압력은 단위가 다르지만, 그것만으로 클래스를 나누지는 않습니다. 같은 단위 칸에 다른 값을 채우면 되니까요. 반면 값을 읽는 채널과 명령을 받아 움직이는 장치는 필요한 정보도, 연결의 뜻도 처음부터 다릅니다. 이런 차이는 같은 칸의 값만 바꿔서는 표현할 수가 없어요.

[SLIDE 21] 정보를 표현하는 네 가지 선택

실제 자료를 받아 보면 대상 이름, 부품 번호, 회사 이름, 납기가 한꺼번에 섞여 나옵니다. 이 정보를 각각 어디에 놓을지, 이 표 하나로 골라 보겠습니다.

둘째 줄, 속성부터 볼게요. 쿨러 코어의 부품 번호는 그 부품 하나에 딸린 값입니다. 그러니까 부품의 속성이죠. 공급사가 승인된 곳인지도 그 공급사에 딸린 참, 거짓 값이라서 공급사의 속성입니다.

그런데 쿨러 코어 자체는 번호 하나로 끝나지 않아요. 오일 쿨러에 쓰이고, 공급사와도 이어지는 대상이죠. 그래서 셋째 줄처럼 부품 인스턴스로 두고, 오일 쿨러에서 사용하는 부품이라는 관계로 연결합니다. 만약 오일 쿨러의 속성 칸에「쿨러 코어」라고 글자로만 적어 뒀다면, 그 글자에서 공

급사로 이어 갈 길이 없었겠죠. 공급사도 마찬가지예요. 자기 이름과 승인 여부가 있고 다른 부품도 공급하니까 별도 인스턴스가 됩니다.

그럼 납기는 어디에 둘까요? 부품과 공급사를 함께 정해야 며칠 걸리는지가 정해지죠. 그래서 마지막 줄, 둘 사이 관계의 속성으로 둡니다. 그리고 여러 부품이 함께 따를 정의, 여러 공급사가 함께 따를 정의는 맨 윗줄처럼 각각 클래스로 정하고요.

앞에서 본 공급 조건 한 줄로 직접 갈라 볼까요. 펌프 축 씰 키트를 A정밀에서 35만원에, 2일 만에 받는다는 줄이 있다고 해 보죠. 펌프 축 씰 키트는 자기 부품 번호를 가진 부품 인스턴스예요. A정밀도 승인 여부를 가진 공급사 인스턴스고요. 승인된 공급사라는 표시는 A정밀 하나에 딸린 값이라 공급사의 속성입니다. 남은 단가와 납기는 씰 키트와 A정밀을 함께 골라야 정해지니까, 둘을 잇는 공급 관계의 속성이 되죠. 한 줄에 섞여 온 정보가 이렇게 각자의 자리로 나뉩니다.

결국 정보를 받으면 먼저 그게 무엇을 가리키는지 보면 됩니다. 한 대상의 값인지, 자기 정보가 있는 다른 대상인지, 두 대상 사이의 조건인지요. 그걸 가리면 속성, 관계, 관계의 속성 가운데 어느 자리에 놓을지가 정해집니다. 클래스는 그렇게 정리한 대상들이 함께 쓸 정의고요.

이제 이렇게 구별한 대상과 값, 연결을 여러 사람이 함께 따를 정의, 곧 스키마로 적고, 식별자와 제약으로 지키는 법을 보겠습니다.

[이론 02] 스키마 설계

[SLIDE 22] 스키마와 식별자

네 번째부터 여섯 번째 단원에서는 앞에서 구별한 대상과 값, 연결을 여러 사람이 함께 쓸 정의로 적어 봅니다. 먼저 네 번째 단원에서는 함께 지식을 넣을 때 무엇을 맞춰야 하는지부터 보겠습니다. 넣는 사람마다 칸을 다르게 채운다고 생각해 보세요. 같은 대상을 다른 이름으로 등록하기도 하고요. 그러면 같은 지식이 다른 뜻으로 읽히고, 필요한 정보도 여기저기 흩어집니다. 무엇을 미리 정해 둬야 누가 넣어도 같은 뜻으로 읽힐까요? 함께 따를 정의인 스키마, 그리고 대상을 가리키는 식별자에서 답을 찾아보겠습니다.

[SLIDE 23] 스키마와 인스턴스 데이터

센서를 등록하는 장면으로 구체적으로 보죠. 한 사람은 태그를 적었고, 다른 사람은 센서 이름만 적었습니다. 이걸 읽는 프로그램은 어느 칸에서 계측 데이터를 찾아야 할까요? 결국 등록할 때부터 다 같이 따를 정의가 있어야 합니다.

그 정의가 화면 왼쪽의 스키마예요. 앞 회차에서 본 온톨로지의 종류, 속성, 관계, 제약을 실제로 적어 둔 거죠. 어떤 클래스가 있고, 클래스마다 어떤 속성과 관계를 갖고, 어떤 조건을 지켜야 하는지를 적습니다. 왼쪽 칸의 「센서에는 태그가 있어야 한다」, 이 문장이 스키마에 해당하죠. 특정 센서의 태그를 적은 게 아니라, 모든 센서가 따를 조건을 정한 겁니다.

오른쪽은 인스턴스 데이터입니다. 정의에 맞춰서 실제 대상과 그 사이의 관계를 적은 내용이에요. 유온 센서를 하나 등록하고 그 태그를 채우면 이쪽에 들어갑니다. 같은 기종의 설비가 한 대 더 들어와도, 공통 정의가 같다면 늘어나는 건 인스턴스 데이터뿐이에요.

양식과, 그 양식으로 작성한 문서를 떠올리면 쉽습니다. 양식은 쓸 칸을 정하고, 작성된 문서는 그 칸을 채우죠. 다른 점도 있어요. 스키마는 값의 칸만이 아니라 어떤 대상끼리 연결할 수 있는지도 정합니다. 센서가 상태 변수를 읽는다는 관계까지 함께 정의하는 거죠.

이런 약속이 없으면 같은 장치가 여러 이름으로, 같은 관계가 여러 이름과 방향으로, 같은 값이 여러 형식으로 들어갑니다. 화면 메시지에 있는 의미 불일치가 이거예요. 그런데 프로그램은 자기가 아는 모양만 찾거든요. 그러면 지식의 일부를 놓치게 되죠. 이 시스템에서는 에이전트도 그렇게 읽는 쪽이에요. 에이전트가 일을 받을 때 이 스키마의 설명이 함께 주어집니다. 어떤 클래스와 관계가 있는지 알아야, 어떤 이름과 방향으로 찾을지 정할 수 있으니까요. 스키마는 넣는 사람과 읽는 프로그램이 같은 구조를 쓰게 하는 기준입니다.

[SLIDE 24] 식별자 · 이름 · 별칭

정비 기록에는 쿨러, 설비 목록에는 오일 쿨러, 또 다른 문서에는 열교환기라고 적혀 있습니다. 다 같은 장치를 가리킨다면, 이름이 셋이라고 대상을 셋 만들 필요는 없겠죠. 만약 셋으로 따로 만들면 어떻게 될까요? 쿨러에 달린 센서를 찾을 때 그중 한 대상에 붙은 것만 나옵니다. 나머지 둘에 붙은 지식은 놓치는 거죠. 그래서 대상 자체를 가리키는 값과, 사람이 부르는 이름을 나눠 둡니다.

첫 카드가 식별자입니다. 개별 대상 하나를 구별하는 고유한 값이에요. 카드에 적힌 문자열들이 그 역할을 합니다. 콜론 앞부분을 보면 설비, 센서, 상태 변수마다 앞머리가 다르죠. 그래서 유온 센서와 유온은 뒷부분이 같아 보여도, 식별자만 보면 다른 종류의 대상이라는 게 드러납니다. 이름을 고쳐도 같은 대상을 찾을 수 있도록, 식별자는 바꾸지 않아요.

두 번째 카드는 이름입니다. 사람이 읽는 표시죠. 유온 센서, 오일 쿨러 같은 거요. 이름은 겹칠 수도 있고, 고칠 수도 있습니다.

세 번째 카드는 별칭이에요. 같은 대상을 부르는 다른 이름들의 목록입니다. 오일 쿨러에는 쿨러와 열교환기가 들어 있고, 유온에는 오일 온도, 작동유 온도 같은 말이 들어 있죠. 사람마다 표현이 달라도, 이 목록과 대조하면 같은 대상을 찾는 데 도움이 됩니다. 실제로 이 그래프에서 「작동유 온도」로 이름을 검색하면, 첫 결과로 유온이 나와요.

예를 들어 센서 이름을 좀 더 알아보기 쉽게 바꿨다고 해 보죠. 그래도 센서의 식별자는 그대로입니다. 그 센서가 유온을 읽는다는 연결도 그대로 남고요. 이름으로 검색하던 쪽은 새 이름을 알아야 하지만, 식별자로 찾는 쪽은 같은 대상을 계속 찾을 수 있어요.

별칭 목록 끝에 TS1도 들어 있죠. 센서의 태그와 같은 글자입니다. 하지만 태그는 유온 센서라는 계측 채널의 칸에 채운 값이고, 이 별칭은 유온이라는 물리량을 찾는 단서예요.

정리하면 별칭은 찾는 단서고, 식별자는 대상을 구별하는 기준입니다. 그러니까 글자가 비슷하다고 센서와 물리량을 하나로 합치지는 않아요. 유온 센서에서 출발했다면 읽는 관계를 건너서 유온을 찾고, 유온의 별칭으로 출발했다면 물리량에서 바로 기준을 읽는 겁니다.

[SLIDE 25] 여러 저장소를 잇는 공통 식별자

유온의 한계는 찾았습니다. 이번엔 경보가 났을 때 실제 온도가 몇 도였는지 찾으려고 해요. 그런데 계측값의 이력은 다른 저장소에 있습니다. 그러니까 어느 설비의 어느 계측값인지 알아볼 공통 정보가 필요하죠.

화면 왼쪽이 지식 그래프입니다. 설비 코드와 센서의 태그가 있죠. 오른쪽 위 시계열 표도 계측값마다 설비 코드와 태그를 붙여 둡니다. 이 두 값을 맞추면 같은 계측 채널의 이력을 찾을 수 있어요. 거기서 경보가 난 시각을 고르면 그때의 계측값 한 건이 나옵니다. 오른쪽 위에 보이는 경보 시각의 유온 값이 바로 그 한 건이에요.

오른쪽 아래 업무 DB는 설비를 같은 설비 코드로 기록합니다. 여기서 설비 자체를 찾을 때는 센서 태그가 필요 없어요. 찾으려는 게 설비인지, 특정 시각의 계측값인지에 따라 필요한 조건이 달라지는 거죠.

앞에서 지식 그래프는 같은 기종의 구성 요소와 센서 노드를 공유한다고 했죠. 그래서 센서 태그만으로 계측값을 고르면 어느 설비의 값인지 구별이 안 됩니다. 설비 코드까지 함께 써야 하는 이유예요. 두 조건으로 채널을 고르고, 시각으로 값 한 건을 고르는 순서입니다. 쿨러 사건에서 원인을 가릴 때 읽은 냉각 효율 평균도 같은 방식으로 찾았어요. 그래프의 증거가 어느 값을 어떻게 읽을지 알려 주면, 시계열 저장소에서 설비 코드와 태그로 그 채널을 골라 최근 구간의 값을 읽은 거죠.

표의 열 이름도 한번 보세요. 그래프에서 태그라고 부르는 값이 시계열 표에서는 이름 열에 들어가 있어요. 열 이름이 같아서 이어지는 게 아닙니다. 같은 계측 채널을 뜻하도록 맞춰 둔 값이 같아서 이어지는 거죠. 그래서 스키마에 다른 시스템의 코드를 받아 두면, 지식에서 찾은 대상으로 실제 데이터까지 조회할 수 있습니다. 대신 그 값은 한 글자라도 다르면 이어지지 않아요.

이게 단원 처음 물음의 답이에요. 함께 따를 정의와 대상을 가리키는 식별자가 있어야, 누가 넣은 지식이든 같은 대상으로 읽히고 다른 저장소와도 이어집니다.

[SLIDE 26] 속성과 연결의 정의 · 제약

함께 따를 정의와 식별자를 정했으면, 다음은 그 정의를 얼마나 자세히 적느냐입니다. 다섯 번째 단원의 물음은 이거예요. 데이터가 잘못 들어왔다는 걸 무엇을 기준으로 알아볼 수 있을까요? 답은 정의를 어떻게 적어 두느냐에 있습니다. 센서를 예로 들어서, 어떤 대상인지, 어떤 값을 반드시 가져야 하는지, 어떤 대상과 연결할 수 있는지를 차례로 적어 보겠습니다.

[SLIDE 27] 클래스 이름 · 의미 · 속성 정의

화면은 센서 클래스의 정의입니다. 이름으로 클래스를 구별하고, 설명으로 범위를 정하고, 속성 목록에 채울 항목을 적어요. 위에서부터 이 순서로 읽어 보죠.

맨 위에는 프로그램에서 쓰는 영문 이름과, 사람이 읽는 한글 이름이 있습니다. 그 아래 설명은 센서를 계측 채널이라고 정의하고 있어요. 왜 이런 설명까지 적을까요? 센서라는 말만 있으면, 물리적인 계기만 포함하는지 계산한 값을 내놓는 채널도 포함하는지, 사람마다 다르게 이해할 수 있거든요.

계측 채널이라고 범위를 정해 뒀으니, 직접 재는 유온 센서도, 계산해서 내놓는 냉각 효율 채널도 같은 클래스에 들어갑니다. 앞에서 본 두 인스턴스를 모두 담도록 범위를 정의한 거죠.

그 아래 목록은 한 줄이 속성 하나입니다. 태그 줄을 볼게요. 속성 이름을 적고, 자료형은 문자열로 정하고, 반드시 있어야 한다고 표시했습니다. 식별자와 이름도 필수예요. 단위와 계산 채널 여부에는 필수 표시가 없죠.

설명의 둘째 문장도 보세요. 태그 이름이 실시간 데이터와 연결된다고 적혀 있죠. 앞에서 본 공통 식별자가 바로 이 태그예요. 정의에 이렇게 적어 두면, 태그 칸이 왜 필수인지도 함께 전해집니다.

이렇게 적어 두면 등록하는 사람은 무엇을 채워야 하는지 알고, 검사하는 쪽은 빠진 값을 찾을 수 있습니다. 화면은 지금 필요한 부분만 뽑은 거예요. 전체 정의에는 계층이나 준용 표준 같은 항목도 있지만, 지금 읽을 핵심은 이름과 의미, 속성입니다.

[SLIDE 28] 자료형과 허용값

유온을 한계와 비교하려는데, 값이 숫자가 아니라 글자로 들어와 있다면 어떨까요? 사람은 읽을 수 있어도, 프로그램은 그대로 크기를 비교하기 어렵습니다. 그래서 속성마다 값의 종류, 곧 자료형을 정해요.

화면에는 다섯 가지를 골랐는데, 스키마에는 이 밖에 정수와 날짜 · 시각을 뜻하는 자료형도 쓰입니다. 위에서부터 볼게요. 이름이나 태그는 문자열입니다. 대상을 부르는 글자니까 계산할 일이 없죠. 반면 정상값이나 한계는 수로 둡니다. 잰 값과 비교하거나 차이를 계산해야 하니까요. 단위는 숫자에 붙여서 한 문자열로 만들지 않고 따로 칸에 적어요. 둘째 줄에 단위를 섞은 글자는 수로 바로 비교할 수 없다고 적어 둔 게 그 얘기입니다.

세 번째 참, 거짓은 둘 중 하나인 값이에요. 계산 채널인지, 승인된 공급사인지처럼 여부를 나타낼 때 쓰죠. 네 번째 목록은 별칭처럼 이름이 여러 개 필요한 속성에 씁니다. 문자열 하나와 문자열 여러 개의 목록은 같은 모양이 아니에요.

마지막 열거형은 미리 정해 둔 허용값 가운데 하나만 받습니다. 화면에서는 상태 변수의 종류가 여기에 해당해요. 팬 속도처럼 직접 바꾸는 조작 변수, 유온처럼 결과로 나타나는 변수, 오염도처럼 바깥에서 영향을 주는 교란 변수, 이 셋 중 하나입니다. 이 설비에서 조작 변수로 적힌 건 화면

마지막 줄처럼 팬 속도, 펌프 부하, 운전 펌프 선택, 셋이에요. 구동기가 값을 써서 바꾸는 변수들이죠.

조작할 수 있는 변수만 골라내려면 이 종류가 일관되게 적혀 있어야 합니다. 사람마다 다른 말로 채우면, 뜻이 같아도 조회에서 빠질 수 있거든요. 열거형은 쓸 수 있는 표현을 제한해서 이런 차이를 줄여 줍니다. 그러니까 자료형이 값의 모양을 정한다면, 열거형의 허용값은 그 안에서 무엇을 고를 수 있는지까지 정하는 거예요.

[SLIDE 29] 필수 속성과 선택 속성

값의 종류를 정했으면, 다음은 그 값을 꼭 채워야 하는지입니다. 표에서 가운데 열이 필수 속성, 오른쪽 열이 선택 속성이에요. 미리 말씀드리면, 빈칸이 있다고 다 잘못된 데이터는 아닙니다.

가운데 열의 단위와 종류는 여섯 행 모두에 채워져 있죠. 단위를 모르면 온도인지 압력인지 값의 뜻을 해석할 수가 없고, 종류를 모르면 조작할 값인지 결과로 볼 값인지 구별할 수 없습니다. 그래서 필수로 정했어요. 센서의 태그도 같은 이유로 필수입니다. 계측 데이터를 찾는 데 쓰는 항목인데, 빠지면 그 일을 못 하니까요.

오른쪽 열의 정상값과 한계는 선택입니다. 먼저 토출 압력 행을 보세요. 한계가 정상값보다 낮죠. 압력은 낮아지는 쪽이 위험해서 한계가 정상값 아래에 있습니다. 앞 회차에서 본 인터록 세 가지의 방향 그대로예요. 유온과 진동은 높은 쪽, 압력은 낮은 쪽을 봅니다.

위쪽의 유온, 압력, 진동에는 이렇게 한계가 있지만, 아래쪽 냉각 효율, 유량, 펌프 부하에는「없음」이라고 돼 있죠. 여기서 기억해 둘 게 있어요. 없음은 한계가 영이라는 뜻이 아닙니다. 이 속성에 정해진 값이 없다는 뜻이에요. 빈칸을 영으로 채우면, 없던 한계를 새로 정한 것처럼 뜻이 바뀌어 버립니다.

냉각 효율 행은 경보 조건과 구별해서 봐야 합니다. 냉각 효율이 낮은지는 쿨러 성능 저하 경보를 판단할 때 쓰여요. 그런데 그 경보의 조건과 이 표의 한계는 서로 다른 기준입니다. 경보에 숫자가 쓰였다고 해서 그 숫자를 상태 변수의 빈 한계 칸에 옮겨 넣지는 않아요. 채우지 않아도 되는 항목과, 꼭 필요한데 빠진 항목을 구별하는 것, 그게 필수 여부를 정의하는 이유입니다.

[SLIDE 30] 관계 정의와 유효한 연결

속성뿐 아니라 연결에도 조건이 필요합니다. 화면 위쪽은 센서가 상태 변수를 읽는 관계의 정의예요. 아래 네 줄은 같은 뜻을 나타내려고 그은 선들이고요. 이 선들이 정의에 맞는지 하나씩 확인해 보겠습니다.

먼저 정의를 보죠. 관계 이름, 시작 클래스, 끝 클래스, 카디널리티가 적혀 있습니다. 센서에서 상태 변수 쪽으로 향해야 하고, 여러 센서가 같은 상태 변수를 읽을 수 있다는 뜻이에요.

아래 네 줄을 유온 센서, 유온, HYD-01로 읽어 볼게요. 첫 줄은 유온 센서가 유온을 읽는다는 연결이라 이 조건을 그대로 따릅니다. 둘째 줄은 유온 센서가 HYD-01을 읽는다는 꼴이라 끝이 설비고요. 셋째 줄은 유온이 유온 센서를 읽는다고 시작과 끝을 뒤집었고, 넷째 줄은 같은 뜻을 스키마에

없는 이름으로 적었습니다. 사람은 뜻을 짐작할 수 있죠. 하지만 프로그램은 정의된 이름과 방향으로 찾습니다. 비슷한 뜻이라고 이름을 마음대로 바꾸면 같은 관계로 찾을 수가 없어요.

이렇게 정의에 맞는 연결을 유효한 연결이라고 합니다. 그럼 클래스만 맞추면 실제 사실도 맞는 걸까요? 맨 아래 가정을 보세요. 유온 센서를 팬 진동에 이었습니다. 시작은 센서, 끝은 상태 변수니까 클래스 조건에는 맞아요. 그런데 재는 물리량을 잘못 골랐죠.

둘째부터 넷째 줄 같은 오류는 스키마와 대조하면 찾을 수 있습니다. 하지만 이 마지막 오류는 센서가 실제로 무엇을 재는지 원자료와 대조해야 찾을 수 있어요. 그래서 연결 조건을 검사한 다음에, 기대한 대상까지 제대로 이어졌는지도 따로 확인합니다. 검사가 어디까지 답해 주는지 알고 결과를 읽어야 하는 거죠.

[SLIDE 31] 스키마의 제약과 데이터 오류

정의한 조건을 실제 데이터와 대조하면 어떤 오류를 찾을 수 있을까요? 화면의 카드 네 장은 조건과, 그 조건에 어긋난 예를 짝지어 놓은 겁니다. 카드마다 위 줄이 조건, 아래 줄이 오류예요. 이런 조건을 제약이라고 합니다.

첫 카드, 태그 없는 센서는 필수값이 빠진 경우입니다. 두 번째 카드는 값의 형식이나 선택이 틀린 경우예요. 숫자로 정한 한계에 단위를 섞은 글자를 넣었거나, 종류 항목에 허용하지 않은 값을 넣은 거죠. 세 번째는 같은 식별자로 센서를 또 만든 경우, 그러니까 대상을 중복으로 적은 겁니다. 마지막 카드는 관계를 거꾸로 긋거나 정의에 없는 이름을 쓴 경우, 곧 연결 조건이 틀린 거고요.

이 네 경우는 사람이 그냥 읽으면 뜻을 알아볼 수도 있습니다. 그런데 프로그램이 정의대로 쓰려고 하면 문제가 생겨요. 태그로 데이터를 못 찾거나, 수를 비교하지 못하거나, 엉뚱한 대상을 찾게 되죠.

마지막 카드의 이름 문제는 정의 없이 여러 사람이 넣을 때 실제로 잘 생깁니다. 설비가 구성 요소를 갖는다는 같은 사실을, 누구는 「갖는다」로, 누구는 「속한다」로, 누구는 「포함한다」로 적는 거죠. 그러면 정의된 이름으로 찾는 쪽은 그중 하나만 찾고, 다른 이름으로 적힌 연결은 놓칩니다. 연결 조건이 있으면 정의에 없는 이름을 바로 가려낼 수 있어요.

다만 조건을 적는 일과 실제로 검사하는 일은 구별해야 합니다. 어떤 조건을 누가 언제 강제하는지는 그래프 DB를 다룰 때 따로 보겠습니다.

누가 강제하든 기준은 정의에서 나옵니다. 앞 회차에서는 제약을 올바른 연결의 조건으로 봤는데, 이번 단원에서 보니 연결만이 아니죠. 속성의 자료형과 필수 여부, 허용값, 관계의 시작과 끝을 정의로 적어 두면, 그 조건이 모두 제약이 되고 어긋난 데이터를 가려내는 기준이 돼요. 단원 처음에 물었던, 잘못 들어온 데이터를 무엇으로 알아보느냐의 답이 바로 이겁니다.

[SLIDE 32] 설비 모델의 설계와 확장

정의를 적는 법과 제약을 알았으니, 여섯 번째 단원에서는 설비 자료에서 그 정의를 직접 만들어 봅니다. 설계는 어디서부터 시작해야 할까요? 클래스 이름부터 잔뜩 적기보다, 이 모델로 답하려는 질문을 먼저 정하는 게 출발점입니다. 그 질문에서 대상과 값, 연결을 골라 설비 모델을 만들고, 새 질문이 생기면 어디를 고치는지도 보겠습니다.

[SLIDE 33] 질문에서 클래스 · 속성 · 관계로

이 설비의 상태를 어떤 센서로 확인할 수 있을까요? 그 센서가 읽는 물리량의 한계는 어디에 있을까요? 쿨러를 정비한다면 어떤 교체 부품이 필요할까요? 화면 첫 칸에 적은 이 세 질문이 이번 설계의 목표입니다. 이렇게 온톨로지가 답할 수 있어야 하는 질문을 역량 질문이라고 합니다. 필요한 지식을 빠뜨리지 않았는지 확인할 기준을 질문으로 먼저 세우는 거예요.

두 번째 칸으로 가 보죠. 첫 질문에는 설비, 구성 요소, 센서가 필요합니다. 둘째 질문에는 센서가 읽는 물리량과 그 한계가 필요하고, 셋째 질문에는 구성 요소에 쓰는 부품이 필요하죠. 그래서 대상의 종류는 다섯으로 정리됩니다.

그런데 종류만 정해서는 아직 답을 찾을 수 없어요. 설비와 쿨러, 센서와 유온을 실제 예로 놓고, 질문마다 길을 이어야 합니다. 설비에서 구성 요소로, 거기서 센서로 이어지면 계측 채널을 찾을 수 있고요. 센서에서 물리량으로 한 번 더 가면 그 물리량의 한계에 닿습니다. 교체 부품은 구성 요소에서 다른 길로 연결하고요.

세 번째 칸이 속성입니다. 한계는 물리량의 속성이에요. 비교에 쓸 수 있게 수로 정하고, 각 대상은 식별자로 구별합니다. 마지막 칸이 관계예요. 이름과 방향, 시작과 끝의 클래스, 연결 수를 정합니다.

질문을 먼저 정하면 넣을 범위도 선명해집니다. 자료에 설비의 색이나 무게가 있더라도, 이 세 질문에 쓰이지 않는다면 지금 모델에 다 옮길 필요는 없어요. 반대로 자료에 값이 아무리 많아도, 센서에서 물리량으로 가는 관계를 빠뜨리면 한계를 찾는 질문에는 답하지 못합니다.

이 질문들은 설계가 끝난 뒤에도 다시 씁니다. 예시 데이터를 넣고 질문에 답해 봤을 때, 원자료에서 기대한 대상과 값이 나와야 하거든요. 처음 정한 질문이 설계의 범위이면서, 결과를 확인하는 기준이 되는 거죠.

[SLIDE 34] 설비 모델의 속성과 관계 정의

질문에서 골라낸 대상과 값을 이제 정의로 적었습니다. 위쪽이 클래스의 속성이고, 아래쪽이 클래스 사이의 관계예요. 전체 정의에는 정상값이나 계산 채널 여부 같은 속성도 더 있지만, 화면에는 세 질문에 필요한 핵심만 뽑았습니다. 각 줄이 어느 질문 때문에 들어갔는지 맞춰 보겠습니다.

중괄호 안에서 느낌표가 붙은 속성이 필수예요. 식별자와 이름은 다섯 클래스 모두에 필요합니다. 설비 코드와 센서 태그는 실제 계측 데이터를 찾아갈 때 쓰는 값이고요. 상태 변수의 단위와 종류는 물리량을 해석할 때 필요하죠. 한계는 느낌표 없이 선택 속성으로 둡니다.

아래 관계 네 줄 중 처음 둘은 설비의 센서를 찾는 길입니다. 세 번째는 센서에서 물리량을 찾아 한계를 읽는 길이고, 마지막은 구성 요소에서 교체 부품을 찾는 길이에요. 같은 클래스라도 어떤 관계를 정의하느냐에 따라 답할 수 있는 질문이 달라집니다.

구성 요소와 센서 사이는 다대다로 정했어요. 쿨러 하나를 유온과 냉각 효율 두 채널로 볼 수 있고, 센서 하나도 여러 구성 요소의 상태를 보는 데 연결될 수 있으니까요. 그리고 설비와 구성 요소 사이는 일대다로 선언했는데, 공통 노드를 공유하는 실제 데이터와는 다르다는 점, 앞에서 봤죠. 화면 오른쪽 마지막 칸에 그 점도 함께 적어 뒀습니다. 정의를 읽을 때부터 실제 모델이 나타내는 범위를 놓치지 않으려는 거예요.

[SLIDE 35] 설비 모델의 관계 구조

글로 적은 정의를 그림으로 옮겼습니다. 가운데 진한 원, 구성 요소에서 길이 갈라지죠. 이 그림에서 센서의 한계를 찾는 길과 교체 부품을 찾는 길을 각각 직접 따라가 보세요.

같이 볼게요. 왼쪽 위 설비에서 가운데 구성 요소로 온 다음, 오른쪽 위 센서로 갑니다. 거기서 오른쪽 상태 변수로 가면 한계가 적힌 대상에 닿죠. 교체 부품을 찾을 때는 구성 요소에서 아래쪽 부품으로 가면 됩니다. 두 질문 모두 같은 쿨러에서 출발하지만 쓰는 관계는 서로 달라요.

원의 색도 이 구조를 보여 줍니다. 구성 요소에 바로 붙은 설비, 센서, 부품은 파란 테두리고, 상태 변수만 바깥 회색이죠. 상태 변수는 구성 요소에 직접 붙지 않고, 센서를 한 번 더 거쳐야 닿는 대상이라서 그래요.

그리고 원 안에는 실제 설비 이름이 아니라 클래스 이름이 들어 있습니다. 어떤 종류가 어떤 관계로 이어질 수 있는지를 나타낸 그림이에요. 실제 데이터를 채울 때 이 구조에 설비, 쿨러, 센서 같은 인스턴스가 들어갑니다.

설계에서 빠뜨린 연결도 이 그림으로 확인할 수 있어요. 센서에서 상태 변수로 가는 선을 빼 보세요. 계측 채널까지는 찾지만, 그 물리량의 한계에는 닿지 못합니다. 부품 쪽 선을 빼면, 센서 정보가 아무리 자세해도 교체품 질문에는 답을 못 하고요. 질문마다 필요한 길이 끝까지 이어지는지, 이 관계 그림으로 확인하는 겁니다.

[SLIDE 36] 인스턴스 추가와 스키마 변경

모델을 쓰다 보면 설비가 더 들어오기도 하고, 새 질문이 생기기도 합니다. 그런데 이 두 경우에 고치는 자리가 달라요. 화면 왼쪽과 오른쪽이 그 두 경우입니다.

왼쪽은 기존 정의에 대상을 더하는 경우예요. 같은 기종의 새 설비라면, 설비 인스턴스를 하나 더 하고 구성 요소와 연결하면 됩니다. 센서와 물리량, 교체 부품으로 가는 기존 길은 그대로 쓸 수 있고요.

오른쪽은 지금 만든 다섯 클래스 모델에 없는 질문입니다. 쿨러 코어를 어디서 공급받을 수 있는지 알고 싶어요. 부품까지는 닿는데, 공급사를 찾을 정의가 없죠. 그래서 공급사 클래스와, 부품에서 공급사로 가는 관계를 더합니다. 그러면 등록된 두 공급사를 찾을 수 있어요.

두 칸 아래 꼬리표도 보세요. 왼쪽은 자주 일어나는 일, 오른쪽은 드물게, 신중하게 하는 일이라고 적혀 있죠. 스키마는 그 클래스에 속한 모든 대상이 따르는 기준이라서, 한 줄만 고쳐도 이미 들어 있는 데이터 전체가 새 기준으로 다시 판정되거든요. 그래서 스키마를 바꿀 때는 필요한 변경인지, 기존 데이터에 어떤 영향을 주는지 먼저 살핍니다. 이 시스템에서는 정해 둔 정의를 바탕으로 새 지식을 인스턴스로 더하는 방식을 씁니다. 그렇다고 정의를 영원히 바꾸지 않는다는 뜻은 아니에요. 답해야 할 질문에 지금의 정의가 충분한지를 보고 변경 여부를 판단합니다.

새 대상을 추가하는 것과 기존 조건을 바꾸는 것도 구분해야 합니다. 오른쪽 칸 아래처럼, 센서의 단위를 선택에서 필수로 바꾼다고 가정해 볼게요. 전에는 단위 없는 센서도 조건에 맞았습니다. 그런데 바꾼 뒤에는 같은 데이터가 필수값 누락이 되죠. 이미 저장한 센서도 확인해서, 실제 자료에 맞는 단위를 채워야 합니다.

그러니까 스키마를 고칠 때는 어떤 인스턴스와 질의가 영향을 받는지 같이 살펴야 해요. 공급사를 새로 연결하는 경우와 센서의 필수 조건을 바꾸는 경우는 영향 범위가 다르니까요. 질문이 늘었다고 기존 정의를 다 다시 만들지도 않고, 빈 결과가 나왔다고 바로 새 클래스를 더하지도 않습니다. 정의가 빠졌는지, 데이터가 빠졌는지, 질의를 다르게 적었는지부터 확인하는 거예요.

[SLIDE 37] 설비 · 진단 · 판단 · 성과의 지식

설비 모델에서 정의한 방식은 진단과 조치에도 똑같이 쓰입니다. 화면의 카드 네 장이 온톨로지의 네 묶음이고, 묶음마다 답하는 질문이 달라요.

첫 카드, 설비 묶음은 무엇으로 이루어졌는가를 담습니다. 우리가 지금까지 연결한 설비와 구성 요소, 센서와 물리량, 부품이 여기 있어요.

두 번째 진단 묶음은 왜 잘못됐는가를 찾는 구조입니다. 쿨러 사건의 증상에서 고장 유형으로 가고, 원인 후보와 증거를 찾는 클래스들이 여기 있죠.

세 번째 판단 묶음에는 조치 방법과 판단 규칙이 있고, 네 번째 성과 묶음에는 목표와 성과 지표가 있습니다. 카드에 적힌 영어 이름들은 그 대상들을 정의한 클래스 이름이에요. 지금 다 외울 필요는 없습니다. 각 묶음이 어떤 질문에 답하는지만 구별하면 돼요.

묶음 사이도 관계로 이어집니다. 쿨러 사건 하나를 따라가도 네 묶음을 차례로 지나가요. 오일 쿨러와 유온은 설비 묶음에 있고, 증상에서 고장 유형, 원인 후보와 증거로 가는 길은 진단 묶음에 있습니다. 고장 유형은 구성 요소에서 발생한다는 관계로 설비 묶음과 이어져 있고요. 원인에 맞는 조치 방법은 고장 유형과 연결돼 판단 묶음에서 찾고, 그 조치가 무엇을 좋아지게 하는지는 성과 묶음에서 찾습니다. 묶음 사이의 관계가 하나라도 끊기면, 경보에서 출발한 길이 조치까지 닿지 못하겠죠. 설비 모델에서 길을 정의했던 것처럼, 진단과 판단에서도 필요한 질문에 답할 길을 정의해 둔 겁니다. 다루는 범위는 달라도 방법은 같아요. 답할 질문에서 출발해서 클래스, 속성, 관계, 제약을 정하는 거죠.

이어서 이렇게 정의한 지식을 그래프 DB에 저장하고, Cypher 쿼리로 찾고 만드는 법을 보겠습니다.

[이론 03] 그래프 저장과 조회

[SLIDE 38] 그래프 DB의 저장 구조와 Cypher 쿼리 조회

지식을 어떻게 정의할지 정했으니, 이제 그래프 DB에 저장하고 Cypher 쿼리로 찾고 만드는 법을 보겠습니다. 일곱 번째 단원의 물음은 이겁니다. 설계한 대상과 연결은 그래프 DB 안에서 어떤 모습이 될까요? 그리고 그 모습에서 필요한 정보는 어떤 문장으로 찾을까요? 저장 구조를 먼저 보고, 그 구조에 맞춰 Cypher 쿼리로 조회 문장을 적어 보겠습니다. 쿼리 문법을 외우는 게 목표가 아니에요. 질문에 맞는 패턴을 적고, 나온 결과가 정말 그 질문의 답인지 확인하는 게 목표입니다.

[SLIDE 39] 테이블의 조인과 그래프의 연결

쿨러 코어를 공급하는 회사를 찾으려고 합니다. 화면 왼쪽 표와 오른쪽 그래프는 같은 부품, 같은 공급 관계를 서로 다르게 담은 모습이에요.

왼쪽이 관계형 DB입니다. 표 하나를 테이블, 그 안의 기록 한 줄을 행이라고 해요. 위쪽 부품 테이블에는 부품 번호가 있고, 아래쪽 공급사 테이블에는 회사마다 공급하는 부품 번호가 있습니다. 두 표에서 부품 번호가 같은 행끼리 맞추면 공급사를 찾을 수 있죠. 이렇게 열 값을 기준으로 행을 잇는 걸 조인이라고 합니다.

오른쪽 그래프 DB에는 부품 노드에서 공급사 노드 두 개로 관계가 저장돼 있어요. 공급 관계를 따라가기만 하면 같은 두 회사가 나옵니다. 그러니까 왼쪽은 부품 번호가 같은 행을 맞추는 방식이고, 오른쪽은 노드 사이의 관계를 따라가는 방식인 거죠.

여기서 오해하기 쉬운 게 있어요. 표에도 연결 정보는 있습니다. 아래 표의 부품 번호가 어느 부품인지 가리키고 있으니까요. 그래프에만 연결이 있고 표에는 없다, 이렇게 이해하면 안 됩니다. 연결을 표현하고 조회하는 방식이 다른 거예요.

이 그림에서는 부품 하나에 공급사 둘이 대응합니다. 표에서 부품 번호가 두 번 나온다고 부품이 둘인 건 아니죠. 그래프에서는 부품 노드 하나에 선이 둘 붙어 있어서, 같은 사실을 다른 모양으로 보여 줍니다. 어느 방식으로 저장하든 질문의 답은 같은 두 공급사여야 해요.

[SLIDE 40] 노드 · 레이블 · 관계 · 속성

설계한 센서와 유온을 여기서 쓰는 그래프 DB에 넣으면 이렇게 보입니다. 원 하나가 노드고요, 원에 붙인 종류 이름표가 레이블입니다.

왼쪽 위가 유온 센서 노드예요. 센서 클래스의 인스턴스니까 Sensor라는 레이블을 붙였습니다. 옆 상자를 보면 식별자, 이름, 태그가 속성으로 들어 있죠. 속성 이름과 값이 짝을 이루고 있습니다.

아래쪽은 유온 노드입니다. 상태 변수 레이블을 붙이고, 오른쪽 상자처럼 한계와 단위 같은 값을 속성으로 저장했어요. 그리고 두 노드는 읽는 관계로 이어져 있습니다. 관계에는 유형 이름과 방향이 함께 저장돼요.

앞에서 설계할 때 쓴 말과 맞춰 보면 쉽습니다. 인스턴스를 저장한 게 노드고, 클래스 이름은 레이블이 됩니다. 대상을 잇는 관계, 이름과 값의 짝인 속성도 그대로 저장돼요. 공급 조건처럼 관계에 붙는 속성은 관계 쪽에 저장하고요.

레이블은 여러 개 붙일 수도 있습니다. 규정 검토 작업에 작업과 흐름 노드, 이름표 두 개를 같이 붙였던 게 그 예예요. 노드 하나가 두 종류에 속한다는 표시죠. 다만 상속의 정의 자체는 스키마에 있습니다. 이름표 두 개만 보고 데이터베이스가 그 규칙을 알아내는 건 아니에요.

[SLIDE 41] 온톨로지 스키마와 DB 제약

데이터베이스에 저장이 됐다면, 스키마의 조건을 다 지켰다는 뜻일까요? 오른쪽 예로 확인해 보겠습니다. 센서 식별자에는 유일 제약이 걸려 있어요. 그래서 같은 식별자의 센서를 또 만들면 저장이 거절됩니다.

오른쪽 칸의 DB 제약은 데이터베이스가 저장할 때 직접 강제하는 조건입니다. 여기서 유일 제약은 같은 레이블의 노드끼리 식별자가 겹치지 않게 하죠. 센서가 이미 있는지 확인하고 중복 저장을 막는 거예요.

왼쪽 스키마는 훨씬 넓은 조건을 정의합니다. 필수 속성, 허용값, 관계의 시작과 끝, 연결 수까지 정해 두죠. 그런데 이 정의를 적었다고 전부 DB 제약이 되지는 않습니다.

예를 들어 태그가 없는 센서를 넣으면 저장 자체는 될 수 있어요. 이 누락은 저장된 데이터를 스키마와 대조하는 별도 검사가 찾아냅니다. 허용되지 않은 종류 값이나 잘못된 끝 클래스도 이 검사에서 확인하고요.

그런데 이 시스템의 검사기도 스키마의 모든 조건을 보는 건 아닙니다. 왼쪽 칸 맨 아래 적혀 있듯이, 자료형 전체와 카디널리티는 자동으로 대조하지 않아서 따로 확인해야 해요. 그러니까 이 화면의 두 칸은, 무엇을 정의했는지와 무엇을 실제로 강제하는지를 나눠서 읽어야 합니다. 저장 성공 메시지만 보고 검증이 끝났다고 할 수 없는 이유가 여기 있어요.

[SLIDE 42] Cypher 쿼리와 실행 화면의 역할

이제 그래프에 원하는 걸 요청하는 문장을 적어 보겠습니다. 데이터베이스에 보내는 요청을 질의, 또는 쿼리라고 해요. 쿨러 사건에서 경보로부터 원인 후보를 찾아낸 것도 사실 그래프에 보낸 질의였습니다. 여기서 배울 언어는 Cypher예요.

두 번째 줄을 보죠. Cypher 쿼리는 Cypher라는 언어로 적은 질의예요. Cypher는 찾거나 만들 노드와 관계의 모양을 패턴으로 적습니다. 그 칸 마지막 줄의 예처럼, 쿨러에 이어진 센서를 찾고 싶다면 구성 요소에서 센서로 가는 연결 모양을 그대로 적는 거예요. 그러면 데이터베이스가 저장된 그래프에서 그 모양에 맞는 걸 찾아 줍니다.

쿼리를 적고 실행해서 결과를 보는 데는, 맨 아래 줄처럼 Neo4j Browser 같은 화면을 씁니다. 여기서 언어와 실행 화면은 구별해 주세요. 쿼리는 요청의 내용이고, 실행 화면은 그 요청을 보내고 결과를 받아 보는 창구예요.

포털의 지식 지도에서도 그래프를 볼 수 있죠. 그런데 하는 일이 다릅니다. 지식 지도는 항목을 골라서 기준 지식의 속성과 연결을 따라가 보는 화면이에요. 쿼리 실행 화면은 직접 조건을 적어서 조회하거나, 허용된 범위에서 데이터를 만들 때 씁니다. 둘 다 그래프를 보여 준다고 같은 작업 공간은 아닌 거죠.

[SLIDE 43] 노드 · 관계 · 변수 · 속성 조건의 표기

위쪽 연결 그림을 아래쪽 Cypher 표기로 옮겨 봤습니다. 유온 센서에서 출발해서, 읽는 관계를 따라 상태 변수로 가는 모양이에요. 그림과 글자를 위아래로 맞춰 보겠습니다.

둥근 괄호 하나가 노드입니다. 왼쪽 괄호에서 콜론 뒤가 센서라는 레이블이고, 오른쪽 괄호는 상태 변수 레이블이에요. 가운데 대괄호에는 관계 유형을 적고, 양옆의 선과 화살표로 방향을 나타냅니다.

콜론 앞에 있는 짧은 글자는 변수예요. 이 쿼리 안에서 찾은 노드를 다시 부르려고 붙인 이름입니다. 뒤에서 그 변수에 점을 찍고 속성 이름을 붙이면, 그 노드의 속성값을 읽을 수 있어요. 실제 대상의 고유 식별자와는 역할이 다릅니다.

왼쪽 괄호 안의 중괄호는 속성 조건입니다. 센서 전부가 아니라, 태그가 TS1인 센서만 찾도록 좁히는 거죠. 레이블로 종류를 고르고, 속성 조건으로 그 안에서 대상을 더 좁힌 겁니다.

하나 주의할 게 있어요. 태그 조건으로 찾은 결과가 지금은 하나라고 해도, 그 조건이 고유성을 보장하는 건 아닙니다. 특정 대상 하나를 계속 가리켜야 할 때는 식별자를 조건으로 써요. 쿼리를 적을 때는 그림의 양 끝과 방향을 먼저 적고, 필요한 만큼만 조건으로 좁힌다, 이렇게 생각하면 따라가기 쉽습니다.

[SLIDE 44] MATCH · RETURN: 패턴과 결과의 행

쿨러를 계측하는 센서를 찾는 쿼리입니다. 그런데 결과를 보면, 쿨러는 하나인데 이름이 두 번 나와요. 왜 그런지 위쪽 패턴과 아래 결과를 맞춰 보죠.

MATCH는 찾을 패턴을 적는 절입니다. 이름이 오일 쿨러인 구성 요소에서, 계측 관계로 이어진 센서를 찾으라는 거예요. RETURN은 돌려받을 항목을 정하는 절이고요. 여기서는 구성 요소 이름과 센서 태그를 적었으니까 결과도 두 열입니다.

결과 첫 행은 쿨러와 유온 센서의 연결이에요. 둘째 행은 같은 쿨러와 냉각 효율 채널의 연결이고요. 두 연결이 각각 패턴에 맞으니까 결과가 두 행이 된 겁니다. 쿨러 노드가 두 개라서 이름이 반복된 게 아니에요.

쿼리 어디에도 센서 개수를 적지 않았다는 점도 보세요. 쿨러에서 계측 관계로 이어진 센서를 모두 찾으라고만 했죠. 그 조건에 맞는 연결이 실제로 둘이라서 두 행이 나온 겁니다. 한 행이 무엇을 나타내는지 알고 있으면, 결과가 여러 행이라는 것만 보고 중복 데이터라고 판단하지 않게 됩니다.

[SLIDE 45] RETURN: 속성값과 노드 · 관계

지금처럼 이름과 태그 목록이 필요하다면 속성값을 받으면 됩니다. 그런데 연결된 모양을 눈으로 확인하고 싶다면 무엇을 받아야 할까요? 화면 아래쪽의 RETURN만 바꿔 보겠습니다.

위쪽은 이름과 태그, 속성값을 받습니다. 결과 표의 칸에는 글자가 들어가죠. 아래쪽은 구성 요소 노드, 계측 관계, 센서 노드 자체를 받아요. 관계도 다시 부를 수 있게 변수를 하나 붙였습니다.

찾는 패턴은 같으니까 결과는 여전히 두 행입니다. 다만 행마다 담긴 내용이 달라졌죠. 아래 결과를 그림으로 그리면, 쿨러 노드 하나에서 센서 둘로 선이 이어진 모양이 됩니다. 각 행에 들어 있는 쿨러가 같은 노드이기 때문이에요.

속성값만 받은 표를 그래프로 바꾼다고 원래 관계가 새로 생기지는 않습니다. 연결을 그리려면 그릴 노드와 관계를 받아야 해요. 목록을 보고 싶은지, 연결 구조를 보고 싶은지에 따라 반환할 항목을 고르는 겁니다. MATCH가 찾을 범위를 정한다면, RETURN은 그중에서 무엇을 꺼내 볼지 정하는 거죠.

[SLIDE 46] DISTINCT: 반환한 값의 중복 제거

다시 속성값을 받는 쪽으로 돌아가서, 이번엔 채널 목록 말고 장치 이름만 받고 싶다고 해 보죠. 이름 열만 남기면 쿨러도 한 번만 나올까요? 표의 첫 줄을 보세요. 이름만 받았는데도 같은 이름이 두 번 남습니다.

MATCH가 찾은 연결은 여전히 둘이거든요. 결과에서 채널 태그만 뺐을 뿐, 같은 값의 행을 하나로 합치라고 하지는 않았습니다. 이렇게 같은 값은 한 번만 보고 싶을 때 DISTINCT를 써요. 둘째 줄처럼 반환 절에 붙이면 반복된 쿨러 이름이 한 번만 나옵니다.

그럼 셋째 줄은 어떨까요? 이름 옆에 센서 태그도 같이 받았어요. 두 행 모두 쿨러 이름은 같지만 태그는 다르죠. DISTINCT는 이름 열만 따로 보는 게 아니라, 반환한 열 전체의 조합을 봅니다. 쿨러와 냉각 효율 채널의 짝, 쿨러와 유온 센서의 짝은 서로 다르니까 두 행이 그대로 남아요.

DISTINCT를 붙인다고 그래프에 저장된 센서나 관계가 지워지는 건 아닙니다. 지금 돌려받는 결과만 정리하는 거예요. 그리고 이름이 같은 별개의 장치가 있다면, 이름 목록만으로는 둘을 구별하지 못하겠죠. 대상 자체를 구별해야 할 때는 이름 대신 노드나 고유 식별자를 기준으로 삼습니다.

어떤 장치가 있는지 목록만 볼 때와, 장치마다 어떤 채널이 붙어 있는지 확인할 때는 필요한 정보가 다릅니다. 장치 이름 목록만 필요하면 반복된 이름을 줄여도 되고요. 채널별 연결을 보려면 태그까지 남겨야죠. 결과를 짧게 만드는 것보다, 질문에 필요한 구별을 남기는 게 먼저입니다.

[SLIDE 47] COUNT: 결과 행과 서로 다른 대상의 수

목록 말고 개수가 필요할 때도 있죠. 이 결과에 구성 요소는 몇 개 있을까요? 행을 세면 둘인데, 두 행 모두 같은 쿨러를 가리켰죠. 무엇을 세느냐에 따라 답이 달라집니다.

COUNT는 여러 행을 개수 하나로 요약하는 집계 함수예요. RETURN의 첫 번째 식은 괄호 안에 별표가 들어 있죠. 결과로 넘어온 행을 전부 셉니다. 쿨러와 채널을 잇는 연결이 두 개 나왔으니까 행 수는 둘이에요.

두 번째 식은 DISTINCT를 붙여서 서로 다른 구성 요소 노드를 셉니다. 같은 쿨러가 두 행에 들어 있어도 대상은 하나니까 하나로 세요. 세 번째 식은 서로 다른 센서 노드를 셉니다. 유온 센서와 냉각 효율 채널이 있으니까 이쪽은 둘이고요.

아래 결과의 세 열은 같은 그래프에서 서로 다른 세 질문에 답한 겁니다. 첫 열은 몇 행인가, 가운데는 구성 요소가 몇 개인가, 마지막은 채널이 몇 개인가. AS 뒤에 적힌 말은 그 결과 열의 별명이에요. 계산 결과를 알아보기 쉽게 붙인 이름일 뿐, 쿨러나 센서에 속성을 새로 만든 게 아닙니다.

지금은 행 수와 채널 수가 둘로 같죠. 그렇다고 두 값의 뜻까지 같은 건 아닙니다. 한 대상이 여러 연결에 참여하면 결과의 여러 행에 다시 나타날 수 있어요. 쿨러가 바로 그 예였고요. 그래서 서로 다른 대상이 몇 개인지 물을 때는 대상을 구별해서 셉니다.

채널 수가 필요하다면 마지막 식을 고르면 되겠죠. 가운데 식과 마지막 식은 모양이 거의 같습니다. 세는 대상만 구성 요소에서 센서로 바뀌었어요. 그러니까 개수를 구할 때는 먼저 질문이 무엇을 세라고 하는지 확인하고, 그 대상을 가리키는 변수를 세면 됩니다.

[SLIDE 48] Cypher 쿼리: 여러 관계를 잇는 경로 조회

설비의 센서와, 그 센서가 읽는 물리량의 한계를 한 번에 찾아보겠습니다. 설계할 때 그렸던 길을 쿼리에도 그대로 이어 적으면 돼요.

출발점은 설비 코드로 고른 설비입니다. 거기서 구성 요소를 찾고, 계측 센서로 가고, 센서가 읽는 상태 변수까지 갑니다. 노드와 관계를 번갈아 적어서 길 하나를 만든 거예요.

RETURN에는 이 길에서 필요한 속성을 적었습니다. 어느 구성 요소인지, 센서 태그는 뭔지, 그 물리량의 이름과 단위, 한계는 얼마인지를 받죠. 그 아래 ORDER BY는 결과를 태그 순서로 정렬하는 부분이에요.

결과에서 유온 행 하나를 읽어 볼게요. 위에서 네 번째 줄입니다. 이 설비의 오일 쿨러는 유온 센서로 계측되고, 그 센서는 유온을 읽습니다. 끝의 두 칸이 유온의 단위와 한계죠. 앞에서 유온에 붙여 둔 그 한계, 65도가 그대로 나옵니다. 이렇게 경로를 따라 찾은 대상의 속성값까지 한 행에서 읽을 수 있습니다.

냉각 효율과 유량 행의 한계 칸에는 NULL이 보이죠. 앞에서 표에 「없음」으로 봤던 그 빈 한계가, 쿼리 결과에서는 NULL로 나옵니다. 두 물리량까지는 찾아갔는데, 한계 속성에 값이 적혀 있지 않은 거예요. 연결이 끊겨서 대상에 못 닿은 것과는 다르죠. 이걸 구별해서 읽으면, 쓸데없이 관계를 새로 만드는 일을 피할 수 있습니다.

[SLIDE 49] WHERE: 조건과 값의 존재 여부

센서에서 물리량까지 찾아갔습니다. 이제 그중에서 한계 값이 있는 물리량만 남겨 보죠. 찾은 행을 조건으로 거르는 절, 그게 WHERE입니다.

쿼리 둘째 줄의 조건을 보세요. 한계 속성에 값이 있는지를 묻고 있죠. IS NOT NULL은 값이 없는 경우를 빼라는 뜻이에요. 그래서 결과에는 토출 압력, 유온, 팬 진동이 남고, 냉각 효율과 유량은 맨 아래처럼 빠진 행이 됩니다.

빠진 두 행은 센서와 물리량을 못 찾아서 빠진 게 아닙니다. 둘 다 찾았고, 그다음에 한계 칸이 비어 있어서 걸러진 거예요. 이 조건이 묻는 건 값이 있느냐지, 값이 얼마냐가 아니거든요.

크기를 비교하는 조건도 적을 수는 있어요. 오른쪽 마지막 칸처럼 한계가 100보다 큰 것을 고르는 식이죠. 문법으로는 됩니다. 하지만 결과의 단위 열을 보세요. 압력과 온도와 진동은 단위부터 다릅니다. 단위가 다른 물리량의 한계를 숫자 하나로 견주면 뜻이 없는 비교가 돼요. 그래서 이 쿼리는 단위도 함께 받았습니다. 값이 있는지를 묻는 조회와, 그 값으로 무엇을 판단하는지는 따로 생각해야 합니다.

[SLIDE 50] AND · OR: 조건을 괄호로 묶기

한계가 있는 물리량 가운데서 쿨러와 팬에 연결된 것만 더 골라 보겠습니다. 정리하면 쿨러나 팬 중 어느 하나에 연결된 채널이면서, 한계가 있는 물리량이에요. 장치는 둘 중 하나면 되고, 한계 값은 반드시 있어야 하죠. 이렇게 조건이 여럿이 되면 모두 맞아야 하는 것과 하나만 맞으면 되는 것을 나눠서 적어야 합니다.

위쪽 WHERE의 괄호 안을 보세요. 쿨러라는 조건과 팬이라는 조건 사이에 OR가 있죠. 어느 하나만 맞아도 된다는 뜻이에요. 쿨러에 연결됐든 팬에 연결됐든 이 묶음을 통과합니다.

괄호 밖에는 AND로 한계 값 조건을 붙였어요. 장치 조건도 맞고, 한계도 있어야 남는다는 거죠. 그래서 결과에는 유온과 팬 진동이 나오고, 쿨러에 연결돼 있어도 한계가 없는 냉각 효율은 빠집니다.

그런데 맨 아래, 같은 문장에서 괄호만 뺀 결과를 보세요. 냉각 효율이 들어와 있죠. 왜 이렇게 됐을까요? Cypher는 괄호가 없으면 AND를 OR보다 먼저 묶습니다. 그래서 쿨러에 연결됐거나, 또는 팬에 연결되면서 한계가 있는 것, 이렇게 읽혀요. 앞의 쿨러 조건만 만족하면 뒤의 한계 조건과 상관없이 남는 거죠.

냉각 효율 행 하나를 직접 따라가 보면 차이가 보입니다. 장치는 쿨러니까 OR의 앞 조건은 맞아요. 한계 값은 없고요. 원래 쿼리에서는 장치 조건을 통과한 뒤에 한계 조건에 걸려서 빠집니다. 괄호 없는 쿼리에서는 쿨러라는 조건 하나로 남고요.

중요한 건, 두 문장 모두 오류 없이 실행된다는 거예요. 그러니까 오류 표시만 봐서는 질문에 맞는 결과인지 알 수 없습니다. 조건을 쓸 때는 먼저 말로 읽어 보세요. 쿨러 또는 팬을 고르고, 그중에

서 한계가 있는 걸 남긴다. 이렇게 함께 판단할 부분을 괄호에 담으면, 말로 정한 범위와 쿼리의 범위를 맞출 수 있습니다.

[SLIDE 51] 구성 요소별 집계와 ORDER BY 정렬

설비를 살펴보면서, 장치마다 상태를 몇 개 채널로 보고 있는지 비교해 보려고 합니다. 설비 전체 채널 수 하나만으로는 어느 장치에 뭐가 붙어 있는지 모르겠죠. 그래서 구성 요소마다 개수를 따로 세 보겠습니다.

쿼리 위쪽은 이미 읽어 본 길이에요. 설비에서 구성 요소로, 거기서 센서로 갑니다. 오른쪽에 적은 원래 행을 보면, 쿨러에는 냉각 효율과 유온, 주 펌프에는 유량과 압력, 팬에는 진동 채널이 연결돼 있죠.

바뀐 건 RETURN입니다. 구성 요소의 식별자와 이름은 그대로 두고, 센서는 COUNT로 셉니다. 이렇게 집계 함수와 일반 항목을 같이 적으면, 집계하지 않은 항목들의 값이 같은 행끼리 묶어서 계산해요. 쿨러 행은 쿨러끼리, 주 펌프 행은 주 펌프끼리, 팬 행은 팬끼리 묶이는 거죠.

식별자를 같이 남긴 이유도 볼게요. 이름만 기준으로 묶으면, 이름이 같은 다른 장치가 한 묶음이 돼 버릴 수 있습니다. 그래서 이름을 보여 주면서도 대상을 구별할 값은 남겨 둔 거예요. 화면 결과에서는 읽기 편하게 식별자 열만 뺐습니다.

센서를 셀 때는 DISTINCT로 서로 다른 센서 노드를 구별해요. 원래 행을 손으로 묶어 세어도 아래 집계와 같아야 합니다. 쿨러와 펌프는 두 채널씩, 팬은 하나죠.

마지막 줄의 ORDER BY는 결과의 순서를 정합니다. 채널 수 뒤의 DESC는 큰 값부터 놓는 내림차순이에요. 수가 같은 장치는 그다음에 적은 식별자 순서, 오름차순으로 놓고요. 그래야 결과를 비교할 때 늘 같은 순서로 읽을 수 있습니다. 정렬한다고 개수가 바뀌거나 그래프의 관계가 옮겨지지는 않아요.

그리고 이 패턴은 센서까지 이어지는 길이 있는 구성 요소만 찾습니다. 채널이 없는 장치는 결과에 안 나와요. 그러니까 아래 표는 모든 장치의 목록이 아니라, 채널이 연결된 장치들끼리의 비교입니다. 개수만 받으면 보기엔 간단하죠. 하지만 무엇을 어느 범위에서 셌는지 확인하려면 원래 행으로 돌아갈 수 있어야 합니다.

[SLIDE 52] 관계의 저장 방향과 탐색 방향

쿨러 냉각 성능 상실의 원인 후보를 찾아보겠습니다. 그림에서 왼쪽 이상 패턴부터 고장 유형까지는 화살표를 따라왔어요. 그런데 오른쪽 원인 쪽 선들은 고장 유형으로 들어오고 있죠. 이 관계를 쿼리에는 어떻게 적어야 할까요?

저장된 사실은 원인이 고장 유형을 일으킨다는 겁니다. 그래서 화살표 머리가 고장 유형 쪽에 있어요. 고장 유형에서 출발해 원인을 찾더라도 이 사실의 방향은 바꾸지 않습니다.

아래 띠 왼쪽 표기를 보세요. 고장 유형을 먼저 놓고, 거기로 들어오는 관계를 적었습니다. 저장된 선을 거슬러서 원인을 찾는 역방향 탐색이에요. 오른쪽은 고장 유형에서 원인으로 나가는 관계를 적었습니다. 그런데 그림에는 그런 관계가 없죠. 그래서 결과가 안 나옵니다.

문제는 오른쪽도 문법상으로는 맞는 문장이라는 거예요. 그래서 오류 메시지 대신 빈 결과를 보게 됩니다. 결과가 없을 때는, 데이터가 정말 없다고 결론 내리기 전에 찾는 모양과 저장된 모양을 대조해 보세요. 시작과 끝, 관계 이름, 화살표 방향을 보는 거죠.

그리고 앞에서 들어오는 선을 거슬러 볼 때 말했듯이, 반대로 찾겠다고 반대 뜻의 관계를 하나 더 저장하지는 않습니다. 같은 사실을 두 번 적어 두면, 한쪽만 고쳤을 때 둘이 어긋날 수 있거든요. 사실의 방향은 그대로 두고 출발하는 쪽만 바꾸면 됩니다. 다만 여기서 찾은 건 원인 후보예요. 이번 사건의 원인이 무엇인지는 연결된 증거의 값과 기준을 대조한 뒤에 가립니다.

단원 처음에 물었던 게 이거였죠. 설계한 지식은 그래프 DB에서 어떤 모습이 되고, 어떤 문장으로 찾느냐. 지식은 정의한 뜻과 방향 그대로 노드와 관계로 저장됩니다. 찾을 때는 그 저장된 모양을 패턴으로 적고, 질문에 맞춰 출발점과 조건, 돌려받을 항목을 고르면 됩니다.

[SLIDE 53] Cypher 쿼리: 노드 · 관계 생성과 병합

지금까지는 저장된 그래프를 읽기만 했죠. 마지막 이론 단원에서는 읽기에서 쓰기로 넘어갑니다. 지식을 그래프에 넣을 때는 조회할 때와 뭐가 달라질까요? 같은 센서를 두 번 만들면 안 되고, 정의한 방향으로 관계를 이어야 하죠. 만든 다음에는 다시 조회해서 의도한 대상과 연결이 남았는지도 확인해야 하고요. 노드와 관계를 만들고 합치는 Cypher 쿼리를 이 순서로 보겠습니다.

[SLIDE 54] CREATE · MERGE · SET의 역할

유온 센서를 등록하는 방법이 두 가지 있습니다. CREATE는 새로 만들고, MERGE는 패턴에 맞는 대상이 있는지 먼저 찾아본 뒤 없을 때만 만들어요.

맨 위 CREATE는 적힌 센서 노드를 새로 만들라는 요청입니다. 같은 게 이미 있는지는 찾아보지 않아요. 화면 맨 아래 오류 메시지가, 같은 식별자의 센서가 이미 있는 상태에서 이걸 실행한 결과입니다. 센서 식별자의 유일 제약에 걸려서 거절된 거죠.

가운데 MERGE는 식별자로 센서를 찾습니다. 있으면 그 노드를 쓰고, 없으면 만들어요. 그다음 SET이 찾았거나 만든 노드에 이름, 태그, 단위를 넣습니다. 대상을 찾는 일과 속성값을 넣는 일을 나눠서 적은 거예요.

이 차이는 자료를 넣다가 중간에 멈춰서 다시 실행해야 할 때 중요해집니다. 이미 들어간 센서를 또 만들면 안 되겠죠. 같은 식별자로 찾아서 그대로 써야 합니다. 이 예처럼 식별자에 유일 제약을 두고 MERGE를 쓰면, 몇 번을 다시 실행해도 같은 대상이 하나로 남아요.

처음 등록하는 새 센서라면 CREATE로 만들어도 됩니다. 하지만 이미 등록한 센서인지 확인하면서 같은 입력을 다시 처리해야 한다면 MERGE가 맞죠. 새 대상을 만드는 일인지, 같은 대상을 다시 다루는 일인지, 그걸 보고 고르면 됩니다.

[SLIDE 55] MERGE의 검색 조건과 SET의 변경값

이미 등록한 유온 센서의 표시 이름을 「온도 센서」로 바꾸려고 합니다. 센서 자체는 같은 대상이에요. 화면의 두 쿼리를 같은 처음 상태에서 각각 실행한다면, 어느 쪽이 이 뜻에 맞을까요? 잠깐 결과를 예상해 보세요.

위 쿼리는 식별자만으로 찾습니다. 같은 센서가 있으니까 그 노드를 찾고, SET으로 이름을 바꾸죠. 노드는 그대로 하나만 남습니다. 다시 실행해도 같은 센서를 찾고요.

아래 쿼리는 식별자와 새 이름을 둘 다 검색 조건에 넣었습니다. 그런데 기존 이름은 아직 유온 센서잖아요. 두 조건을 다 만족하는 노드가 없으니까 MERGE는 새로 만들려고 합니다. 그 순간, 같은 식별자가 이미 있어서 유일 제약이 막아요.

MERGE가 뜻을 이해해서 같은 센서인지 판단해 주는 게 아닙니다. 우리가 적은 패턴이 전부 맞는지만 확인해요. 그러니까 대상을 계속 가리킬 식별자는 검색 조건에, 바꾸려는 이름은 SET에 둬야 합니다.

[SLIDE 56] 기존 노드를 찾아 관계 생성

노드를 등록했으면 이제 그 사이를 이어야겠죠. 화면 위쪽은 오일 쿨러, 유온 센서, 유온을 먼저 찾은 다음 관계 두 개를 만드는 쿼리입니다.

MATCH가 세 대상을 식별자로 찾아서 변수에 담습니다. 그다음 MERGE가 쿨러에서 센서로 계측 관계를, 센서에서 유온으로 읽는 관계를 만들어요. 두 노드 사이에 같은 관계가 이미 있으면 그걸 그대로 씁니다. 화살표 방향은 스키마에서 정의한 방향이고요.

아래쪽은 센서 식별자를 없는 값으로 적은 경우예요. 센서를 못 찾으면 뒤로 넘길 행이 없으니까 관계도 만들어지지 않습니다. 맨 아래 결과 줄에도 만들어진 관계가 없다고 나오죠. 그런데 이게 오류 없이 끝날 수 있어요. 그러니까 실행 완료 표시만 보고 관계가 생겼다고 판단하면 안 됩니다.

그래서 저장한 뒤에는 같은 두 노드 사이를 조회해서 관계가 있는지 확인합니다. 이 쿼리엔 RETURN이 없으니까, 결과 표가 안 보인다는 것만으로는 성공인지 실패인지 가릴 수도 없어요. 어떤 노드를 찾아서 어떤 관계를 만들었는지 다시 조회해서 확인해야 합니다. 방향을 잘못 적어 저장한 경우도 이 대조에서 정의와 다른 연결로 드러나고요.

[SLIDE 57] 정의 · 적재 · 조회 · 검사의 연결

설계한 지식이 제대로 들어갔는지 확인하는 데까지가 한 과정입니다. 화면의 다섯 칸 중 앞 두 칸은 무엇을 넣을지 준비하는 일이고, 뒤 세 칸은 넣고 확인하는 일이에요.

첫 칸, 스키마 파일에는 클래스, 속성, 관계, 제약을 정합니다. 둘째 칸 예시 데이터에는 실제 대상과 연결을 적고요. 셋째 칸에서는 이 둘을 바탕으로, 식별자를 써서 노드와 관계를 적재합니다.

넷째 칸 조회에서는 처음 정한 역량 질문에 답해 봐요. 쿨러를 계측하는 센서가 맞게 나오는지, 물리량의 한계와 교체 부품이 예상대로 나오는지 보는 거죠. 마지막 칸 검사에서는 필수값과 허용값, 관계 끝점 같은 걸 스키마와 대조합니다.

조회와 검사가 찾아내는 잘못은 서로 다릅니다. 스키마가 허용한 종류끼리 연결했더라도, 실제로는 엉뚱한 센서를 골랐을 수 있어요. 이건 원자료와 예상한 답을 대조해야 찾습니다. 반대로 답 한 건이 제대로 나왔다고, 다른 대상의 필수값까지 다 채워졌다고 할 수는 없죠. 질문의 답과 정의의 조건, 이 둘을 함께 확인해야 지식을 넣는 데서 끝나지 않고 쓸 수 있는 지식을 만드는 데까지 갑니다.

[SLIDE 58~59] 실습: 그래프의 관계 조회

지금까지 지식을 클래스와 관계로 정의하고, 그래프 DB에 저장해서 Cypher로 찾고 만드는 데까지 따라왔죠. 이제 그걸 직접 해 볼 차례입니다. 첫 실습에서는 예제 그래프에서 관계를 따라가며 설비의 구성 요소와 센서를 찾아봅니다. 먼저 조회할 대상과 질문을 확인하고, 한 단계 관계와 여러 단계 경로를 Cypher로 적어서 실행해요. 속성값을 받은 표와 연결을 받은 그래프도 나란히 비교해 봅니다.

기본 조회를 마치면 같은 행을 구성 요소별로 묶어 봅니다. 장치마다 서로 다른 계측 채널이 몇 개 연결돼 있는지 구하고, 그 수로 정렬하는 거죠. 앞에서 화면으로만 읽었던 집계를 직접 적어 보는 셈이에요. 핵심은 무엇을 기준으로 묶고, 무엇을 세느냐를 정하는 겁니다.

원래 행을 직접 묶어서 센 개수와 집계 결과가 같고, 이름이 같은 대상을 잘못 합치지 않았다면 성공입니다. 남길 결과물은 조회 질의와 결과, 집계 대조표, 그리고 어느 그래프를 썼는지에 대한 기록이에요.

조회할 때는 이렇게 질문과 결과를 대조하게 됩니다. 이어지는 실습은 같은 연결을 포털에서 찾아보는 활동이에요.

[SLIDE 60~61] 실습: 기준 지식의 연결 탐색

두 번째 실습에서는 포털의 지식 지도로 설비 지식의 연결을 탐색합니다. 첫 실습이 쿼리를 적어 관계를 찾는 활동이라면, 여기서는 항목을 골라 연결을 따라가 볼 거예요. 앞에서 배운 클래스와 속성, 들어오고 나가는 관계가 실제 시스템에서는 어떻게 보이는지 확인하는 자리이기도 해요. 먼저 어느 설비에서 어떤 정보를 찾을지 정하고, 거기에 연결된 구성 요소와 센서를 살펴볼 겁니다.

찾은 항목마다 클래스와 속성, 들어오는 관계와 나가는 관계를 기록합니다. 그 탐색 기록이 이 실습의 결과물이고, 출발한 대상과 도착한 대상이 처음 정한 질문에 맞으면 성공입니다. 하나 구별해 둘 게 있어요. 여기서 보는 기준 지식과, 직접 만드는 실습 그래프는 서로 다른 작업 대상입니다. 한쪽에서 바꾼 게 다른 쪽에 나타나지 않아요.

조회와 탐색으로 이미 만들어진 구조를 살펴본 뒤에는, 직접 설계하는 활동으로 넘어갑니다.

[SLIDE 62~63] 실습: 온톨로지 설계

세 번째 실습은 설비 자료로 온톨로지를 직접 설계하는 활동입니다. 앞의 두 실습에서 볼 그래프도 대상과 연결을 정해서 만든 것이죠. 이번에는 제가 설비 자료를 읽고, 여섯 번째 단원에서 배운 순서대로 모델을 만들겠습니다. 먼저 이 자료로 답할 역량 질문을 정하고, 질문에 필요한 대상과 값을 나눈 다음, 클래스와 속성, 관계와 제약을 정할 거예요.

정한 내용은 요구사항으로 정리해서, 바이브 코딩으로 스키마 파일과 예시 인스턴스 데이터를 만듭니다. 이 두 파일이 결과물이에요. 고유한 식별자와 필수값, 관계 방향이 정의에 맞고, 함께 쓰는 부품과 한 구성 요소의 여러 센서가 원자료대로 빠짐없이 이어졌다면 성공입니다. 기억해 둘 게 하나 있어요. AI가 파일을 만들어 줬더라도, 이게 무엇을 나타내는 모델인지 설명하고 확인하는 건 설계한 사람의 몫입니다.

이렇게 만든 스키마와 예시 데이터는 마지막 실습에서 실제 그래프로 옮길 겁니다.

[SLIDE 64~65] 실습: 그래프 구현과 검증

마지막 실습에서는 세 번째 실습에서 만들 스키마와 예시 데이터를 그래프 DB에 구현하고 검증합니다. 여덟 번째 단원에서 본, 노드와 관계를 만드는 쿼리를 실제로 쓰는 자리예요. 만들 대상과 연결을 확인하고, 바이브 코딩으로 생성한 Cypher 쿼리를 읽어 본 다음 실행합니다.

구현한 뒤에는 중복과 관계의 끝점을 검사하고, 처음 정한 질문에 예상한 답이 나오는지 대조해요. 이때 의미가 틀린 연결 한 건도 같이 살펴봅니다. 스키마를 배울 때 가정으로 봤던 바로 그 연결이에요. 화면 가운데 카드처럼, 오류 사본에서는 유온 센서가 유온이 아니라 팬 진동에 연결돼 있습니다. 클래스와 방향은 정의에 맞지만, 실제로 재는 대상이 잘못 연결된 거죠.

원자료와 조회 결과를 근거로 잘못된 끝점을 찾아서 고치고, 다시 조회한 대상이 원자료와 일치하면 성공입니다. 결과물은 생성 질의와 그래프, 그리고 수정 전후를 적은 검증 기록이에요.

실습의 순서를 보면, 먼저 있는 지식을 찾아보고, 직접 설계한 뒤, 저장한 결과가 질문에 맞는지 확인하게 됩니다. 자료를 내려받아 작업 폴더를 열고, 첫 활동인 그래프 조회부터 시작하겠습니다.

[SLIDE 66] 설계에서 조회까지: 같은 정보를 이어 읽기

설계한 지식을 그래프에 넣고, 원자료와 조회 결과를 대조하는 실습을 마쳤습니다. 이제 설계할 때 나눴던 대상과 값, 연결이 조회에서는 어떻게 쓰였는지 함께 정리해 보겠습니다.

먼저 복습입니다. 쿨러 성능 저하 경보가 났다고 해 보죠. 유온의 한계는 어디서 읽고, 쿨러의 계측 채널은 어떻게 찾을까요? 그 길을 표의 다섯 줄로 따라가 보겠습니다. 표 왼쪽 열의 정보가 가운데 설계에서는 무엇이고, 오른쪽 저장과 조회에서는 어디에 있는지 찾아보세요. 특히 센서의 태그와 유온의 한계, 이 둘이 같은 노드의 값인지가 관건이에요.

같이 맞춰 볼까요. 첫 줄의 태그는 센서 인스턴스의 속성값입니다. 저장할 때는 센서 노드의 태그 속성에 들어가죠. 둘째 줄의 유온은 그 센서가 읽는 물리량이라서 별도 인스턴스고, 별도 노드로 저장됩니다. 둘 사이에 읽는 관계가 있어야 센서에서 유온을 찾을 수 있어요.

셋째 줄의 한계는 유온에 붙은 값입니다. 그러니까 경보를 받고 유온 센서에서 출발해서 한계를 찾으려면, 관계를 한 번 건넌 다음 물리량의 속성을 읽어야 하죠. 한 노드의 칸을 읽는 것과 다른 노드로 건너가는 것, 이 둘을 구별할 수 있으면 쿼리의 패턴도 훨씬 읽기 쉬워집니다.

넷째 줄은 경보가 난 쿨러를 계측하는 센서를 찾는 질문이에요. 구성 요소에서 센서로 가는 관계를 MATCH에 적고, 확인할 항목을 RETURN에 적습니다. 정의에서 정한 길이 조회에서 찾는 길로 그대로 이어진 거죠.

마지막 줄은 쿼리가 찾은 행과 대상의 수를 구별하는 질문입니다. 쿨러가 두 행에 나왔다면 쿨러도 두 개일까요? 아니죠. 연결된 채널이 달라서 두 행이 나왔고, 쿨러는 하나입니다. 행을 셀지, 서로 다른 대상을 셀지에 따라 COUNT 안에 적을 내용을 고르는 거예요.

질문에 답이 안 나올 때도 이 표를 거꾸로 읽어 보면 됩니다. 필요한 대상과 값이 있는지, 그 대상까지 가는 관계가 있는지, 그리고 쿼리가 그 구조를 제대로 찾고 있는지 확인하는 거죠.

[SLIDE 67] 오늘 다룬 핵심 개념과 주요 용어 정리

구조를 되짚어 봤으니, 이번 회차에 나온 용어도 뜻을 한 번 정리하고 가겠습니다. 표에 아홉 개념의 이름과 뜻을 모았어요.

위에서부터 볼게요. 클래스는 공통 특성을 가진 대상의 유형이고, 인스턴스는 그 유형에 속하는 개별 대상입니다. 속성은 대상에 값을 담는 항목이고, 그 항목에 채운 내용이 속성값이에요. 카디널리티는 한 관계의 양쪽에 대상이 몇 개씩 연결될 수 있는지를 나타냅니다.

스키마는 클래스와 속성, 관계와 제약을 정한 공통 정의예요. 식별자는 같은 대상을 하나로 가리키는, 바뀌지 않는 고유한 값이에요. 이름과 달리 고치지 않죠. 레이블은 그래프 DB에서 노드의 종류를 나타내는 이름표입니다.

Cypher 쿼리는 Cypher로 적은 질의 문장입니다. 노드와 관계의 패턴으로, 그래프 DB에서 조회하거나 만들 일을 적어요. 마지막 DB 제약은 데이터베이스가 저장할 때 직접 강제하는 조건입니다.

[SLIDE 68] Review Quiz: 핵심 흐름 되짚어보기

용어까지 정리했으니, 마지막으로 퀴즈 세 문제를 풀어 보겠습니다. 배운 구분을 판단에 직접 쓸 수 있는지 확인하는 자리예요. 보기의 이름을 기억하는 것보다, 왜 그 자리에 정보를 놓아야 하는지 설명할 수 있는지가 중요해요. 문제마다 보기를 읽고 나면 잠깐 멈출 테니, 그때 답을 골라 보세요. 정답과 까닭은 세 문제를 다 본 다음에 확인하겠습니다.

[SLIDE 69] Quiz 1. 스키마와 인스턴스

첫 번째 문제입니다. 공장에 같은 기종의 유압 설비가 한 대 더 들어왔어요. 새 설비 코드도 받았고요. 이 설비를 지식 그래프에 반영하는 방법으로 가장 알맞은 건 무엇일까요?

1번, 새 설비의 이름으로 클래스를 만든다.
2번, 설비 클래스에 넷째 설비라는 속성을 더한다.
3번, 설비 인스턴스를 하나 더하고 구성 요소와 연결한다.
4번, 기존 설비의 코드를 새 설비의 코드로 바꾼다.

[SLIDE 70] Quiz 2. 관계의 속성

두 번째 문제입니다. 같은 씰 키트를 세 공급사에서 받을 수 있는데, 공급사마다 단가와 납기가 다릅니다. 이 조건은 어디에 두는 게 가장 알맞을까요?

1번, 부품과 공급사를 잇는 관계의 속성.
2번, 부품 클래스의 속성.
3번, 공급사 클래스의 속성.
4번, 단가라는 새 클래스의 인스턴스.

[SLIDE 71] Quiz 3. 저장을 막는 조건

마지막 문제입니다. 센서 노드의 식별자에는 유일 제약이 걸려 있어요. 다음 중에서 저장하는 순간 데이터베이스가 직접 거절하는 건 무엇일까요?

1번, 태그가 빠진 센서.
2번, 이미 있는 센서와 식별자가 같은 새 센서.
3번, 센서에서 설비로 그은 읽는 관계.
4번, 종류 항목에 허용되지 않은 값을 넣은 상태 변수.

[SLIDE 72] 정답과 판단 기준

고른 답과 그 까닭을 같이 확인해 보겠습니다. 첫 문제의 정답은 3번이에요. 같은 유형의 설비가 한 대 더 들어온 거니까 인스턴스를 추가하면 됩니다. 기존 설비를 새 설비로 바꾸는 게 아니죠. 그래서 기존 코드는 그대로 두고, 새 설비를 별도 대상으로 등록해서 구성 요소와 연결합니다.

1번처럼 대상 이름으로 클래스를 만들면, 설비가 늘 때마다 공통 정의도 같이 늘어납니다. 2번의 넷째 설비는 모든 설비가 채울 속성도 아니고요. 이 두 보기는 새로 생긴 게 유형인지, 기존 유형의 대상 하나인지를 먼저 보면 가를 수 있어요. 4번은 기존 설비를 가리키던 코드를 바꿔 버리니까, 기존 설비의 계측값과 업무 데이터 대응까지 어긋날 수 있습니다.

두 번째 문제의 정답은 1번입니다. 부품과 공급사를 함께 정해야 단가와 납기가 정해지죠. 부품에만 두면 공급사별 조건이 섞이고, 공급사에만 두면 그 회사가 파는 다른 부품의 조건과 섞여요. 이 모델에서는 조건 자체를 별도 대상으로 추적하지 않고, 공급 관계의 값으로 적습니다.

세 번째 문제의 정답은 2번입니다. 같은 식별자의 센서가 이미 있으면 유일 제약이 새 노드의 저장을 막아요. 나머지 보기도 올바른 데이터는 아닙니다. 하지만 이 DB에서 저장할 때 직접 거절하는 조건은 아니에요. 태그 누락, 관계 끝점, 허용값 위반은 저장된 데이터를 스키마와 대조하는 검사에서 드러납니다.

이번 회차에서는 질문에 필요한 지식을 정의하고, 그래프에 저장한 뒤 답을 확인하는 과정을 살펴봤습니다.

다음 회차에서는 이렇게 연결한 지식을 바탕으로 어떤 조치를 고를지 살펴보겠습니다. 지금 해도 되는 조치인지 먼저 가리고, 가능한 조치들의 효과와 비용을 비교해 볼 거예요. 수고 많으셨습니다. 감사합니다.